

做题总结 123

计算机专业

perper

2026 年 4 月 19 日



目录

1	Codeforces 刷题总结	7
1.1	遇到的问题以及学到的方法	7
1.2	完美的一年	7
1.3	学校排队 (与 LuoguP3741 类似)	7
1.4	向上取整问题, 不用 ceil 函数	8
2	newcode 刷题总结	9
2.1	遇到的问题以及学到的方法	9
2.2	周赛 119	9
2.3	月赛 124	10
2.3.1	B(数组重排)	10
2.3.2	D 区间查询(求因数模板)	10
2.4	周赛 121	11
2.4.1	B 不想被吃掉	11
2.4.2	查找符合条件字符串 (C 三妖精 say subsequence)	11
2.4.3	D01 串大冒险	12
2.5	周赛 125	13
2.5.1	D 小笨的 01 合并	13
2.6	每日一题	13
2.6.1	12.4 质数统计 (筛法列质数表)	13
2.7	其他	14
2.7.1	括号配对	14
2.7.2	map 里面套 map 的用法	15
2.7.3	BGN18 回文日期	15
2.7.4	BGN24 小红的矩阵染色	16
2.7.5	BGN26 小红背单词	17
2.7.6	BGN28 曼哈顿距离的最大化简	18
2.7.7	字符串移动的处理	19
2.7.8	BGN34 01 序列	20
2.7.9	字符串交换	21

2.8	数论总结	22
2.8.1	BGN7 魔法棒	22
2.8.2	同余性质和整除特性的应用	24
2.8.3	环形数组连续 k 项和相等的数论判定推导	26
2.8.4	取模或循环问题周期循环节的寻找	28
2.8.5	裴蜀定理	31
3	算法学习篇 (题目主要来自 Luogu, 但也有部分其他网站的题)	33
3.1	遇到的问题以及学到的方法	33
3.1.1	使用的 main 函数和准备工作都存在这里了	33
3.2	深度优先搜索	33
3.2.1	P11388 飞跃	33
3.2.2	递归实现排列型枚举 (有顺序)	34
3.2.3	递归实现组合型枚举 (无顺序)	35
3.2.4	递归实现指数型枚举	35
3.2.5	指数型变式	36
3.2.6	排列型变式	36
3.2.7	连通块问题	37
3.2.8	P1644 跳马	39
3.2.9	P2036 放调料	40
3.2.10	P1149 火柴棒等式	41
3.2.11	P1036 选数	42
3.2.12	P1135 选和不选类 (多方案选择类)	43
3.2.13	P1025 数的划分	43
3.3	二分答案	44
3.3.1	AcWing789 二分模板 (开区间)(点击此处)	44
3.3.2	P2678 跳石头	45
3.3.3	P1824 进击的奶牛	45
3.3.4	B3629 吃冰棍	46
3.3.5	P1918 保龄球 (二分法)	47
3.3.6	提高组二分答案 (P1083)	48
3.3.7	P1577 切绳子	49

3.3.8	二分进阶 (newcodeF)	50
3.3.9	聪明的质检员	50
3.3.10	P5119 奶牛上车	51
3.3.11	P2650 弹幕考察	52
3.3.12	2026 天梯选拔 2-1	53
3.4	分数规划	54
3.4.1	P1570KC 咖啡	54
3.5	哈希表	55
3.5.1	A+B	55
3.5.2	A-B	56
3.5.3	P1918 保龄球 (哈希表)	56
3.5.4	哈希表解决数对问题	57
3.5.5	哈希数组 (P3879)	58
3.6	双指针	59
3.6.1	盛最多水的容器 (LeetCode)	59
3.6.2	接雨水 (LeetCode)	59
3.6.3	P1147 连续自然数之和	60
3.6.4	P1638 逛画展	60
3.6.5	B4006 宝箱	61
3.6.6	P5745 区间最大和	62
3.6.7	Leetcode3 无重复最长子串	62
3.6.8	Leetcode5 最长回文子串	63
3.7	前缀和以及差分	63
3.7.1	前缀和模板 (p8218)	63
3.7.2	差分模板 (P2367)	64
3.7.3	前缀和解决区间重叠问题 (P1047)	65
3.7.4	P3406 海底高铁	65
3.7.5	P8830(评委打分)	66
3.7.6	二维前缀和模板 (P2004)	67
3.7.7	前缀和取模应用 (abc105D)	68
3.7.8	二维差分 (P3397)	68

3.8	ST 表	69
3.8.1	ST 表模板题 (P3865)	70
3.8.2	ST 表求最小值和最大值 (综合)	71
3.9	动态规划	71
3.9.1	PTA 探险	71
3.10	字符串	72
3.10.1	KMP 算法	72
3.11	有关二进制的习题总结	73
3.11.1	2026 天梯选拔 1-8	73
3.11.2	2025 天梯赛复赛 7-5	74
3.11.3	P1062 数列	75
3.12	快速幂	76
3.12.1	2026 天梯赛选拔 1-7	76
3.13	单调栈	77
3.13.1	P5788 单调栈模板	77
3.13.2	P1318 积水面积	77
3.14	其他 (Luogu 题库)	78
3.14.1	P3741 凑 VK	78
3.14.2	B3627 立方根 (<code>cbrt()</code> 函数)	78
3.14.3	P1765 手机键盘	79
3.14.4	P5734 字符串的连接, 插入, 截取和查找	79
3.14.5	P1957 区分输入个数以及保存上次输入	80
3.14.6	P1614 队列实现循环	81
3.14.7	P1161 开关和向下取整问题	82
3.14.8	负数的进制转换	83
3.14.9	P1061 循环递增字母序列	83
3.14.10	字串成比例数字查找	84
3.14.11	解二元一次方程组 (<code>sscanf</code> 的用法)	84
3.14.12	P5250 set 的用法	87
3.14.13	P1056 排座位	88
3.14.14	P1072 最大公因数 (约数) 和最小公倍数	89

3.14.15 康托尔折线 (z 字形—蛇形矩阵遍历)	90
3.14.16 异或和的特性及用法	90
4 lanqiao 刷题总结	92
4.1 遇到的问题以及学到的方法	92
4.1.1 最少回文子串	92
4.1.2 合理配置	92
5 PTA 刷题总结	93
5.1 遇到的问题以及学到的方法	93
5.2 对数组每一列按升序排序	93
5.3 轮流问题	94
5.4 结构体的使用	94
5.5 固定位数问题	95
5.5.1 字符串查找连续满足条件串的方法	96
5.5.2 活用字符串子串提取函数 (substr)	96
5.5.3 最小化思想 2026 天梯选拔赛 1-6	97
6 AtCoder 刷题总结	99
6.1 遇到的问题以及学到的方法	99
6.2	99
7 AcWing 刷题总结	100
7.1 遇到的问题以及学到的方法	100
7.2 2816 判断子序列	100

1 Codeforces 刷题总结

在本节中，我将总结我在 Codeforces 上做题的经验。

1.1 遇到的问题以及学到的方法

1.2 完美的一年

- 题目链接: [217A\(点击此处\)](#); 方法: 在判断一个字符串是否每个字符均不相等时可以用 **set** 集合来去重，最终比较 **set** 的长度和原字符串的长度来判断是非满足条件；

代码示例：

```
1 void solve() {
2     ll n, num;
3     cin >> n;
4     num = n + 1;
5     while (num) {
6         string s;
7         s = to_string(num);
8         set<char> st(s.begin(), s.end());
9         if (s.size() == st.size()) {
10             cout << s << endl;
11             break;
12         }
13         num++;
14     }
15 }
```

1.3 学校排队 (与 LuoguP3741 类似)

- 题目链接: [266B 学校排队 \(点击此处\)](#)；总结：通过字符串交换来达到目的的题都可以用 **X 填充思想**；

代码示例：

```
1 void solve() {
2     ll n, t;
3     cin >> n >> t;
4     string s1, s2;
5     cin >> s1;
6     s2 = s1;
7     while (t--) {
8         for (ll i = 0; i < n - 1; i++) {
9             if (s1[i] == 'B' && s1[i + 1] == 'G' && s2[i] != 'X') {
10                 swap(s1[i], s1[i + 1]);
11                 s2[i] = 'X';
12             }
13         }
14     }
15 }
```

```
12         s2[i + 1] = 'X';
13     }
14 }
15 s2=s1;
16 }
17 cout << s1 << endl;
18 }
```

1.4 向上取整问题，不用 ceil 函数

- 题目链接: [629A 整除性问题 \(点击此处\)](#);

代码示例:

```
1 void solve() {
2     ll a, b;
3     cin >> a >> b;
4     if (a % b == 0) {
5         cout << 0 << endl;
6     } else {
7         ll c = (a + b - 1) / b; //代替ceil函数实现向上取整即(a+b-1)/b
8         ll d = c * b;
9         cout << d - a << endl;
10    }
11 }
```


2 newcode 刷题总结

在本节中，我将总结我在 newcode 上做题的经验。

2.1 遇到的问题以及学到的方法

2.2 周赛 119

- 题目链接: [B\(点击此处\)](#); 思路: 直接最大的-最小的 (但是要先排序);

代码示例:

```
1 void solve(){
2     int n;
3     cin>>n;B
4     vector<int>a(n);
5     for(int i=0;i<n;i++){
6         cin>>a[i];
7     }
8     sort(a.begin(),a.end());
9     cout<<a.back()-a.front()<<endl;
10 }
```

- 题目链接: [C\(点击此处\)](#); 方法: 当遇到数学问题找满足条件的数字时要学会打表 (先写一个计算机写一个简易的代码逐渐扩大范围找有没有什么规律);

代码示例:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main(){
4     int l,r;
5     cin>>l>>r;
6     for(int i=1;i<=r;i++){
7         if(需要的条件){
8             cout<<i<<endl;
9         }
10    }
11 }
```

最终发现不论在怎样大的范围里只有 1 和 9 满足条件;

2.3 月赛 124

2.3.1 B(数组重排)

- 题目链接: [B 数组重排 \(点击此处\)](#) 总结: 因为 i 是逐渐递增的, 所以只要数组逐渐递减就可以使元素减去下标的值互不相等;

代码示例:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int T;
10    if (!(cin >> T)) return 0;
11    while (T--) {
12        int n;
13        cin >> n;
14        vector<ll> a(n);
15        for (int i = 0; i < n; ++i) cin >> a[i];
16        sort(a.begin(), a.end(), greater<ll>()); // 降序排列
17        for (int i = 0; i < n; ++i) {
18            if (i) cout << ' ';
19            cout << a[i];
20        }
21        cout << '\n';
22    }
23    return 0;
24 }
```

2.3.2 D 区间查询(求因数模板)

- 题目链接: [D 区间查询 \(点击此处\)](#);

代码示例:

```
1 void solve(){
2     cin>>a>>b>>l>>r;
3     ll d,ans=0;
4     d=b-a;
5     for(ll i=1;i*i<=d;++i){
6         if(d%i==0){
7             //因子i;
8             ll x1=b+d/i;
9             if(x1>=l&&x1<=r){
10                 ans++;
11             }
12             //因子d/i(与i成对);
```

```

13         if(i*i!=d){ //避免重复
14             ll x2=b+i;
15             if(x2>=l&& x2<=r){
16                 ans++;
17             }
18         }
19     }
20 }
21 cout<<ans<<endl;
22 }

```

2.4 周赛 121

2.4.1 B 不想被吃掉

- 题目链接: [B 不想被吃掉 \(点击此处\)](#); 总结: 根据贪心可知, 下一天最好先把上一天剩下的先吃完, 因为上一天剩下的这天之后就不能吃了。

代码示例:

```

1 void solve(){
2     ll n,x;
3     cin>>n>>x;
4     vector<ll> v(n+1);
5     for(ll i=1;i<=n;i++){
6         cin>>v[i];
7     }
8     ll prev=0;
9     for(ll i=1;i<=n;i++){
10         ll use=min(prev,x); //先吃上一天剩下的。
11         ll need=x-use; //计算吃掉上一天的后还需要多少。
12         if(v[i]<need){
13             cout<<"No"<<endl;
14             return;
15         }
16         prev=v[i]-need;
17     }
18     cout<<"Yes"<<endl;
19 }

```

2.4.2 查找符合条件字符串 (C 三妖精 say subsequence)

- 题目链接: [C 三妖精 say subsequence](#);

代码示例:

```

1 void solve(){
2     ll n;
3     string s;

```

```

4     cin>>n>>s;
5     vector<ll> v(26);
6     //计算每一个字母的个数实则是看看每个字母可以放几个位置，像数学里的全排列
7     for(auto x:s){
8         v[x-'a']++;
9     }
10    ll ans=0;
11    //三个for循环的作用是保证三个字符两两不同
12    for(ll i=0;i<26;i++){
13        for(ll j=i+1;j<26;j++){
14            for(ll k=j+1;k<26;k++){
15                ans=(ans+v[i]*v[j]*v[k]*6)%MOD; // *6的意思是只有三个字符，
16                //每次对选出来的三个字符全排列的个数是6
17            }
18        }
19    }
20    cout<<ans<<endl;

```

2.4.3 D01 串大冒险

- 题目链接: [D 恋恋的 01 串大冒险 \(点击此处\)](#);

代码示例:

```

1 void solve(){
2     ll n,k;
3     string s;
4     cin>>n>>k>>s;
5     s=' '+s;
6     ll cur_zero=0,i=1;
7     ll cnt=0;
8     while(i<=n){
9         if(s[i]=='1'){
10             if(i<=n&& s[i]=='1'){
11                 i++;
12             }
13             cur_zero=0;
14         }else{
15             cur_zero++;
16             if(cur_zero==k){
17                 v[cnt++]=i-1;
18                 cur_zero=0;
19             }
20             i++;
21         }
22         if(i>n){
23             v[cnt]=i-1;
24             break;
25         }
26     }
27     for(ll j=1;j<=n;j++){
28         v[j]=max(v[j],v[j-1]);
29     }
30     for(ll j=0;j<=n;j++){
31         cout<<v[j]<<" ";
32     }
33 }

```

2.5 周赛 125

2.5.1 D 小苯的 01 合并

- 题目链接: [D 小苯的 01 合并 \(点击此处\)](#); 总结: 用队列寻找字串是一个较为新颖的思想, 可以学习一下。变式题在 [AcWing2816](#);

代码示例:

```
1 auto solve() -> void {
2     ll n, m;
3     cin >> n >> m;
4     string s1, s2;
5     cin >> s1 >> s2;
6     queue<ll> q1, q2;
7     for (ll i = 0; i < n; i++) {
8         q1.emplace(s1[i] - '0');
9     }
10    for (ll i = 0; i < m; i++) {
11        q2.emplace(s2[i] - '0');
12    }
13    while (q1.size() && q2.size()) {
14        if (q1.front() == q2.front()) {
15            q1.pop();
16            q2.pop();
17        } else {
18            const ll x = q1.front();
19            q1.pop();
20            q1.front() ^= x;
21        }
22    }
23    //0和任意数异或都等于那个数, 所以最后如果剩下偶数个1或者若干个0时可以消掉;
24    ll remain = 0;
25    while (q1.size()) {
26        remain ^= q1.front();
27        q1.pop();
28    }
29    if (remain == 0 && q2.size() == 0) {
30        cout << "YES" << endl;
31    } else {
32        cout << "NO" << endl;
33    }
34 }
```

2.6 每日一题

2.6.1 12.4 质数统计 (筛法列质数表)

- 题目链接: [12.4 质数统计 \(点击此处\)](#); 用筛法先把所有质数列成一个表, 然后直接查找; 思想: 若找到一个质数, 那么就把它的所有倍数全部删掉;

代码示例:

```
1 const int N=1e6+10;
2 ll d[N];
3 ll num[N];
4 void solve() {
5     ll q;
6     cin >> q;
7     for (ll i = 2; i < N; ++i) {
8         if (d[i] == 1) continue;
9         num[i]++;
10        for (ll j = i + i; j < N; j += i) {
11            d[j] = 1;
12        }
13    }
14    for (ll i = 1; i < N; ++i) {
15        num[i] += num[i - 1];
16    }
17    while (q--) {
18        ll l, r, cnt = 0;
19        cin >> l >> r;
20        cnt = num[r] - num[l - 1];
21        cout << cnt << endl;
22    }
23 }
```

2.7 其他

2.7.1 括号配对

- 题目链接: [noob84 括号配对问题 \(点击此处\)](#); 用栈进行;

代码示例:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7     int n = s.size();
8     stack<char> st;
9     int b = 1;
10    for (int i = 0; i < n; i++) {
11        if (s[i] == '(' || s[i] == '[') {
12            st.push(s[i]);
13        } else if (s[i] == ']') {
14            if (st.empty() || st.top() != '[') {
15                b = 0;
16                break;
17            }
18            st.pop();
19        } else if (s[i] == ')') {
20            if (st.empty() || st.top() != '(') {
21                b = 0;
22                break;
23            }
24            st.pop();
25        }
26    }
27    cout << b << endl;
28 }
```

```

25     }
26 }
27 if (b)cout << "true" << endl;
28 else cout << "false" << endl;
29 return 0;
30 }

```

2.7.2 map 里面套 map 的用法

- 题目链接: [周赛 123C 小红出对 \(点击此处\)](#)

下面这种遍历方法非常重要，一定要会用!!! ;

代码示例:

```

1 auto solve() -> void {
2     ll n;
3     cin >> n;
4     map<ll, map<char, ll> > mp;
5     for (ll i = 1; i <= n; i++) {
6         ll num;
7         char c;
8         cin >> num >> c;
9         if (mp[num].find(c) == mp[num].end()) {
10             mp[num][c] = i;
11         }
12     }
13     vector<pair<ll, ll> > v;
14     //一定要理解下面这种遍历方式和在这题中的技巧性。
15     for (auto &[num,ch]: mp) {
16         vector<ll> dis;
17         for (auto &[c,id]: ch) {
18             dis.push_back(id);
19         }
20         for (ll i = 0; i + 1 < dis.size(); i += 2) {
21             v.push_back({dis[i], dis[i + 1]});
22         }
23     }
24     cout << v.size() * 2 << endl;
25     for (auto x: v) {
26         cout << x.first << " " << x.second << endl;
27     }
28 }

```

2.7.3 BGN18 回文日期

- 题目链接: [回文日期 \(点击此处\)](#);

代码示例:

```

1 // 月份天数表, 平年情况
2 int days[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

```

```
3
4 // 检查日期是否合法
5 bool check(int date) {
6     int year = date / 10000;
7     int month = date % 10000 / 100;
8     int day = date % 100;
9
10    if (month == 0 || month > 12) return false;
11    if (day == 0) return false;
12
13    // 针对 2 月处理闰年
14    if (month != 2) {
15        if (day > days[month]) return false;
16    } else {
17        bool leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
18        if (day > 28 + leap) return false;
19    }
20    return true;
21 }
22
23 int main() {
24     int date1, date2;
25     cin >> date1 >> date2;
26
27     int ans = 0;
28     // 策略：枚举年份（前四位）
29     for (int i = 1000; i <= 9999; i++) {
30         // 构建回文：把 i 翻转作为后四位
31         // 例如 i = 2021, t = 1202
32         int t = i, date = i;
33         for (int j = 0; j < 4; j++) {
34             date = date * 10 + t % 10;
35             t /= 10;
36         }
37
38         // 判断构造出的日期是否在范围内，且日期本身合法
39         if (date >= date1 && date <= date2 && check(date)) {
40             ans++;
41         }
42     }
43
44     cout << ans << endl;
45     return 0;
46 }
```

总结：要是从其实日期一直遍历到终止日期太过复杂要考虑闰年还有年份天数等多种情况，所以我们就选择遍历所有年份看看对应回文串的日期和天数是否合法即可：要多培养逆向思维!!!

2.7.4 BGN24 小红的矩阵染色

- 题目链接：[矩阵染色 \(点击此处\)](#);

代码示例：


```
1 int main() {
2     int n, m, k;
3     cin >> n >> m >> k;
4     vector<vector<int>> > v(n, vector<int>(m));
5     for (int i = 0; i < n; i++) {
6         for (int j = 0; j < m; j++) {
7             char c;
8             cin >> c;
9             if (c == '*') {
10                 v[i][j] = 0;
11             } else {
12                 v[i][j] = 1;
13             }
14         }
15     }
16     vector<int> sum;
17     for (int j = 0; j < m; j++) {
18         int cnt = 0;
19         for (int i = 0; i < n; i++) {
20             if (v[i][j] == 1) {
21                 cnt++;
22             }
23             if (v[i][j] == 0 && cnt > 0) {
24                 sum.emplace_back(cnt);
25                 cnt = 0;
26             }
27         }
28         //重点注意这一步一定不要遗忘。处理列末尾的情况!!!
29         if (cnt > 0) sum.emplace_back(cnt);
30     }
31     int total = 0;
32     sort(sum.begin(), sum.end(), greater<int>());
33     for (auto x: sum) {
34         if (k <= 0) break;
35         int use = min(k, x);
36         if (use > 0) {
37             total += use - 1;
38             k -= use;
39         }
40     }
41     cout << total << endl;
42     return 0;
43 }
```

2.7.5 BGN26 小红背单词

- 题目链接: [小红背单词 \(点击此处\)](#);

代码示例:

```
1 int main() {
2     int n;
3     cin >> n;
4     set<string> st;
5     unordered_map<string, int> mp;
6     int cnt = 0;
7     while (n--) {
8         string s;
```

```

9      cin >> s;
10     if (st.count(s)) {
11         continue;
12     }
13     mp[s]++;
14     if (mp[s] == cnt + 1) {
15         st.insert(s);
16         cnt++;
17     }
18 }
19 cout << cnt << endl;
20 }

```

2.7.6 BGN28 曼哈顿距离的最大化简

- 题目链接: [最大 FST 距离](#);

最大曼哈顿距离推导

对于平面上的两点 (x_i, y_i) 和 (x_j, y_j) , 其曼哈顿距离定义为:

$$d(i, j) = |x_i - x_j| + |y_i - y_j| \quad (1)$$

为了求得所有点对中的最大距离 $\max_{i,j} d(i, j)$, 我们利用绝对值的性质 $|a| = \max(a, -a)$, 将公式展开。注意: `aligned` 必须放在数学环境内。

$$\begin{aligned}
 |x_i - x_j| + |y_i - y_j| &= \max\{\pm(x_i - x_j) \pm (y_i - y_j)\} \\
 &= \max \begin{cases} (x_i - x_j) + (y_i - y_j) = (x_i + y_i) - (x_j + y_j) \\ (x_i - x_j) - (y_i - y_j) = (x_i - y_i) - (x_j - y_j) \\ -(x_i - x_j) + (y_i - y_j) = -(x_i - y_i) + (x_j - y_j) \\ -(x_i - x_j) - (y_i - y_j) = -(x_i + y_i) + (x_j + y_j) \end{cases} \quad (2)
 \end{aligned}$$

根据上述推导, 最大距离即为:

$$\max_{i,j} d(i, j) = \max \left(\max_i (x_i + y_i) - \min_j (x_j + y_j), \max_i (x_i - y_i) - \min_j (x_j - y_j) \right) \quad (3)$$

代码示例:

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     int n;
6     if (!(cin >> n)) return 0;
7
8     // 初始化极值为极大/极小值
9     ll s_max = -4e18, s_min = 4e18; // 4e18 是为了防止 10^9 平方后溢出
10    ll d_max = -4e18, d_min = 4e18;
11
12    for (ll i = 1; i <= n; i++) {
13        ll a;
14        cin >> a;
15
16        ll x = i * i;
17        ll y = a * a;
18
19        // 计算当前点的 x+y 和 x-y
20        ll current_s = x + y;
21        ll current_d = x - y;
22
23        // 更新四个极值
24        s_max = max(s_max, current_s);
25        s_min = min(s_min, current_s);
26        d_max = max(d_max, current_d);
27        d_min = min(d_min, current_d);
28    }
29
30    // 曼哈顿距离最大值的经典公式
31    ll ans = max(s_max - s_min, d_max - d_min);
32    cout << ans << endl;
33
34    return 0;
35 }

```

2.7.7 字符串移动的处理

- 题目链接: [BGN6 小红的字符串修改 \(点击此处\)](#);

代码示例:

```

1 auto min_pay(const char& a, const char& b) -> ll {
2     ll diff = abs(a - b);
3     return min(diff, 26 - diff);
4 }
5
6 int main() {
7     string s, t;
8     cin >> s >> t;
9     ll n = s.size();
10    ll m = t.size();
11    ll mmin = 1e18;
12    for (ll i = 0; i <= m - n; i++) {

```

```

13     ll curr_pay = 0;
14     for (ll j = 0; j < n; j++) {
15         curr_pay += min_pay(s[j], t[i + j]);
16     }
17     mmin = min(mmin, curr_pay);
18 }
19 cout << mmin << endl;
20 return 0;
21 }

```

- 题目链接: [BGN32 小红的字符串 \(点击此处\)](#); 总结: 对于回文串的问题, 双指针是一个很好的方法去解决这类问题。

代码示例:

```

1 auto min_pay(const char& a, const char& b) -> ll {
2     ll diff = abs(a - b);
3     return min(diff, 26 - diff);
4 }
5
6 int main() {
7     string s;
8     cin >> s;
9     ll len = s.size();
10    ll l = 0, r = len - 1;
11    ll cnt = 0;
12    while (l <= r) {
13        if (s[l] == s[r]) {
14            l++;
15            r--;
16        } else {
17            cnt += min_pay(s[l], s[r]);
18            l++;
19            r--;
20        }
21    }
22    cout << cnt << endl;
23    return 0;
24 }

```

总结: 对于字符串移动所需次数问题, 这两个例子的 min_pay 函数是对于这类题的通用解法, 要理解一下

2.7.8 BGN34 01 序列

- 题目链接: [01 序列 \(点击此处\)](#);

代码示例:

```

1 int main() {
2     ll n;

```

```
3     cin >> n;
4     vector<ll> v(n);
5     for (ll i = 0; i < n; i++) {
6         cin >> v[i];
7     }
8     ll x;
9     cin >> x;
10    ll cnt = 0;
11    for (ll i = 1; i < n; i++) {
12        if (!v[i] && !v[i - 1] && !v[i + 1]) {
13            cnt++;
14            v[i]=1;
15        }
16    }
17    if (cnt >= x) {
18        cout << "true" << endl;
19    } else {
20        cout << "false" << endl;
21    }
22    return 0;
23 }
```

2.7.9 字符串交换

- 题目链接: [BGN38 交换到最大 \(点击此处\)](#);

代码示例:

```
1 int main()
2 {
3     int t; cin >> t;
4     while (t--)
5     {
6         string s; cin >> s;
7         int len = s.size();
8         for (int i = 1; i < len; i++)
9         {
10            int a = i;
11            //不限制操作次数(或者可以多次连续操作)的问题要记得考虑一下while而不是if.
12            while (a > 0 && s[a] - 1 > s[a - 1])
13            {
14                s[a]--;
15                swap(s[a], s[a - 1]);
16                a--;
17            }
18        }
19        cout << s << endl;
20    }
21    return 0;
22 }
```

2.8 数论总结

2.8.1 BGN7 魔法棒

- 题目链接: [魔法棒 \(点击此处\)](#);

魔法棒分裂问题的数论原理分析

1. 核心发现：魔法操作的本质是“增量累加”

当我们将 1 根魔法棒分裂成 k^2 根时，魔法棒总数 n 的变化如下：

$$n_{\text{new}} = n_{\text{old}} - 1 + k^2 = n_{\text{old}} + (k^2 - 1)$$

其中 $k \in \mathbb{Z}^+$ 且 $k \geq 1$ 。这意味着每次操作实质上是为总量增加了一个步长为 $k^2 - 1$ 的增量。

若初始拥有 1 根魔法棒，目标是达到恰好 x 根，则问题转化为：是否存在一组正整数序列 $\{k_1, k_2, \dots, k_m\}$ ，使得：

$$x - 1 = \sum_{i=1}^m (k_i^2 - 1)$$

2. 基础增量的选取：为何只需考虑 3 和 8？

观察不同的 k 取值所产生的增量集合 $S = \{k^2 - 1 \mid k \geq 2\}$ ：

- 当 $k = 2$ 时，增量为 $2^2 - 1 = 3$ ；
- 当 $k = 3$ 时，增量为 $3^2 - 1 = 8$ ；
- 当 $k = 4$ 时，增量为 $4^2 - 1 = 15$ ；
- 当 $k = 5$ 时，增量为 $5^2 - 1 = 24$ 。

由于 $15 = 3 \times 5$ ，且 $24 = 8 \times 3$ ，较大的增量均可由基础增量 $\{3, 8\}$ 组合而成。根据数论中的 **Frobenius Coin Problem**（硬币问题），我们只需关注最小的互质增量组合。

3. 数学原理：模 3 剩余系的“三条跑道”

为了判断 $x - 1$ 能否被 $3a + 8b$ ($a, b \in \mathbb{N}$) 表示，我们将所有整数按 $(\bmod 3)$ 分类：

1. **第一类：** $x - 1 \equiv 0 \pmod{3}$

此类数字形如 $3k$ ，可全部由基础增量 3 凑出。

结论：只要 $x - 1 \geq 0$ 且余数为 0，均可行。

2. **第二类：** $x - 1 \equiv 2 \pmod{3}$

由于 $8 \equiv 2 \pmod{3}$ ，我们可以使用一个 8 来抵消余数，剩下的部分 $x - 1 - 8$ 必为 3 的倍数。

结论：只要 $x - 1 \geq 8$ 且余数为 2，均可行。

3. **第三类：** $x - 1 \equiv 1 \pmod{3}$

由于 $8 + 8 = 16 \equiv 1 \pmod{3}$ ，我们需要至少两个 8 来抵消余数，剩下的部分 $x - 1 - 16$ 必为 3 的倍数。

结论：只要 $x - 1 \geq 16$ 且余数为 1，均可行。

4. 无法构造的“黑名单”数值

根据上述逻辑，无法通过 $3a + 8b$ 构造的 $x - 1$ 集合为：

- 余数为 2 但小于 8 的数：{2, 5}；
- 余数为 1 但小于 16 的数：{1, 4, 7, 10, 13}。

对应到目标总数 x （即 $(x - 1) + 1$ ），最终的不可达成集合（Blacklist）为：

$$x \in \{2, 3, 5, 6, 8, 11, 14\}$$

除此 7 个特例（及 $x < 1$ 的非法输入）外，所有正整数 x 均输出 Yes。

代码示例：

```
1 #include <iostream>
2
3 using namespace std;
4
5 void solve() {
6     long long x;
```

```

7     if (!(cin >> x)) return;
8
9     if (x == 2 || x == 3 || x == 5 || x == 6 || x == 8 || x == 11 || x == 14) {
10        cout << "No" << endl;
11    } else {
12        cout << "Yes" << endl;
13    }
14 }
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(NULL);
19
20     int t;
21     cin >> t;
22     while (t--) {
23         solve();
24     }
25     return 0;
26 }

```

2.8.2 同余性质和整除特性的应用

整数整除性及其数位同余原理分析

1. 核心理论：十进制表示的同余多项式

任意正整数 n 均可表示为 $k+1$ 位十进制形式：

$$n = \sum_{i=0}^k a_i \cdot 10^i = a_k 10^k + a_{k-1} 10^{k-1} + \cdots + a_1 10^1 + a_0$$

其中 $a_i \in \{0, 1, \dots, 9\}$ 。判断 n 是否能被 d 整除，本质上是考察 $n \pmod{d}$ 是否为 0。根据模运算性质，这取决于 $10^i \pmod{d}$ 的表现。

2. 模 3 与模 9：弃九法的数论本质

由于 $10 \equiv 1 \pmod{3}$ 且 $10 \equiv 1 \pmod{9}$ ，则对于任意 $i \in \mathbb{N}$ ，均有 $10^i \equiv 1^i \equiv 1$ 。由此可得：

$$n = \sum_{i=0}^k a_i \cdot 10^i \equiv \sum_{i=0}^k a_i \cdot 1 \equiv \text{SumOfDigits}(n) \pmod{3, 9}$$

结论：一个数与其各位数字之和对 3 或 9 同余（一个数字除以 3（或 9）得到的余数，竟然和它“各位数字加起来的和”除以 3（或 9）得到的余数完全一样。）。

3. 模 2^p 与模 5^p ：末位截断原理

由于 $10 = 2 \times 5$ ，当除数 d 为 2^p 或 5^p 时，对于所有 $i \geq p$ ，有 $10^i \equiv 0 \pmod d$ 。

- 当 $p = 1$ ($d = 2, 5$)： $n \equiv a_0 \pmod d$ 。仅需看最后一位。
- 当 $p = 2$ ($d = 4, 25$)： $n \equiv 10a_1 + a_0 \pmod d$ 。仅需看最后两位。
- 当 $p = 3$ ($d = 8, 125$)： $n \equiv 100a_2 + 10a_1 + a_0 \pmod d$ 。仅需看最后三位。

4. 模 11：奇偶位差判定法

由于 $10 \equiv -1 \pmod{11}$ ，则对于任意 $i \in \mathbb{N}$ ，有 $10^i \equiv (-1)^i \pmod{11}$ 。代入多项式可得：

$$n \equiv \sum_{i=0}^k a_i (-1)^i = a_0 - a_1 + a_2 - a_3 + \dots \pmod{11}$$

结论：一个数能被 11 整除，当且仅当其“偶数位之和”与“奇数位之和”的差能被 11 整除。

5. 编程常用的数论性质归纳

分类	数学性质 / 公式	编程场景
数根定理	$dr(n) = 1 + ((n - 1) \bmod 9)$	$O(1)$ 处理循环数位和
约数对称	$d n \iff (n/d) n$	$O(\sqrt{n})$ 试除法优化
欧拉定理	$a^{\phi(m)} \equiv 1 \pmod m \quad (\gcd(a, m) = 1)$	大数幂模运算、逆元求解
斐波那契	$\gcd(F_n, F_m) = F_{\gcd(n, m)}$	结合 GCD 求解数列性质

6. 同余运算的“避坑” 准则

在编程实现（如 C++）中，模运算需严格遵守：

1. 减法处理：为防止出现负余数，应执行 $(a - b \% m + m) \% m$ 。
2. 除法转换： $(a/b) \pmod m$ 必须转化为 $a \cdot \text{inv}(b) \pmod m$ ，其中 $\text{inv}(b)$ 为 b 的模逆元。

- 题目链接:[九倍平方数 \(点击此处\)](#)

结论：能被 9 整除的数，其各位数字之和也能被 9 整除 (3 也有同样的性质)；

代码示例：

```

1 void solve() {
2     string n;
3     cin >> n;
4     int curr_sum = 0;
5     int count2 = 0, count3 = 0;
6     for (auto x : n) {
7         int diff = x - '0';
8         curr_sum += diff;
9         if (diff == 2) count2++;
10        if (diff == 3) count3++;
11    }
12
13    for (int i = 0; i <= min(count2, 10); i++) {
14        for (int j = 0; j <= min(count3, 10); j++) {
15            if ((curr_sum + 2 * i + 6 * j) % 9 == 0) {
16                cout << "YES" << endl;
17                return;
18            }
19        }
20    }
21    cout << "NO" << endl;
22 }
```

2.8.3 环形数组连续 k 项和相等的数论判定推导

环形数组连续 k 项和相等的数论判定

1. 局部约束向周期性的转化

设环形数组为 $A_0, A_1, \dots, A_{n-1}$ 。给定任意连续 k 个元素的和相等，对比相邻的两个 k 项和：

$$\sum_{j=i}^{i+k-1} A_j \pmod{n} = \sum_{j=i+1}^{i+k} A_j \pmod{n}$$

消去公共项 $A_{i+1}, \dots, A_{i+k-1}$ ，可得：

$$A_i = A_{(i+k)} \pmod{n}$$

这表明数组具有以 k 为步长的周期性。

2. 周期组的划分

在长度为 n 的环上以步长 k 跳跃，根据群论性质，所有下标将被划分为 $d = \gcd(n, k)$ 个独立的轨道（组）。每组包含的元素个数为：

$$L = \frac{n}{\gcd(n, k)}$$

同一轨道内的所有元素必须相等。

3. 频数限制条件

设数组中各不同数值出现的次数（频数）分别为 $c_1, c_2, \dots, c_m$ 。由于每一种数值必须填满整数个轨道，因此每个频数 c_i 必须是单个轨道长度 L 的倍数：

$$L \mid c_i, \quad \forall i \in \{1, \dots, m\}$$

这等价于 L 必须整除所有频数的最大公约数。

4. 最终判定结论：

若使环形数组满足连续 k 项和相等，当且仅当：

$$\frac{n}{\gcd(n, k)} \mid \gcd(c_1, c_2, \dots, c_m)$$

- 题目链接：[洛谷 div3 龙卷风 \(点击此处\)](#)；

代码示例：

```

1  auto solve() -> void {
2      ll n, m;
3      if (!(cin >> n >> m)) return;
4
5      unordered_map<ll, ll> mp;
6      for (ll i = 0; i < n; i++) {
7          ll x;
8          cin >> x;
9          mp[x]++;
10     }
11     ll g = 0;
12     for (auto const& [val, freq] : mp) {

```

```
13         if (g == 0) g = freq;
14         else g =gcd(g, freq);
15     }
16
17     ll cnt = 0;
18     for (ll k = 1; k <= m; k++) {
19         ll d = gcd(n, k);
20         ll L = n / d;
21         if (g % L == 0) {
22             cnt++;
23         }
24     }
25     cout << cnt << endl;
26 }
```

2.8.4 取模或循环问题周期循环节的寻找

知识点详解：动态偏移下的周期性映射 (Periodic Mapping under Dynamic Offset)

核心逻辑

当遇到“数据在数组里不动，但访问规则在转圈”的情况（如加密转盘、循环排班、复杂轨道运动），可采用以下四步法进行抽象。

1. 物理索引与逻辑索引的解耦 (Decoupling)

这是处理循环数据结构的核心，旨在解决“物理存储”与“逻辑步进”之间的映射关系。

- 物理索引 (Physical Index): 数据在内存或数组里的真实下标位置（固定不动）。
- 逻辑索引 (Logical Index): 题目逻辑要求的“第 i 项”。
- 偏移量 (Offset): 逻辑上的“第 0 项”对应物理上的具体下标。

通用映射公式：

$$\text{Index}_{\text{physical}} = (\text{Index}_{\text{logical}} + \text{Offset}) \pmod{N}$$

- 向右循环移动 k 位：偏移量表现为 $-k$ 。
- 向左循环移动 k 位：偏移量表现为 $+k$ 。

2. 复合周期的判定 (LCM Principle)

当一个系统受多个相互独立的周期性变量控制时，系统的总状态周期是各分量周期的最小公倍数 (LCM)。在本模型中：

- 访问位置周期： $i \pmod n \implies T_1 = n$
- 指令位移周期：每 m 天移动一次，移动 n 次回到原位 $\implies T_2 = m \times n$
- 系统总周期： $L = \text{lcm}(T_1, T_2) = \text{lcm}(n, m \cdot n) = m \cdot n$

在这个周期 L 内，系统每天的状态序列（如位移量 $dist$ ）会完全重复。

3. 周期性模拟的“三部曲”框架

对于总天数 t 极大的模拟题，利用周期性进行“倍增跳跃”：

第一步：计算单周期贡献

模拟一个完整周期 L ，记录该周期内累积的总变化量 S ：

$$S = \sum_{i=0}^{L-1} f(i) \pmod D$$

第二步：处理整周期跳跃

利用整数除法快速跨越大部分时间：

$$\Delta_{\text{full}} = (\lfloor \text{TotalDays} / L \rfloor \times S) \pmod D$$

第三步：处理剩余碎片

处理不足一个周期的部分 $R = \text{TotalDays} \pmod L$ ：

$$\text{Result} = (\Delta_{\text{full}} + \Delta_{\text{remaining}}) \pmod D$$

4. 易错点：负数取模的鲁棒性 (Robustness)

在数学上 $-1 \pmod 5 = 4$ ，但在 C++/Java 等编程语言中，表达式 $-1 \% 5$ 的结果通常是 -1 ，直接作为数组下标会导致越界崩溃。

通用防错公式：

$$\text{index} = (a - b \pmod n + n) \pmod n$$

变量含义解析：

- a (基准逻辑索引)：目标对象在逻辑序列中的当前位置 (通常是题目要求的第 i 项)。
- b (动态偏移量)：整个坐标系相对于物理数组起始点的位移。如果系统向右转动了 k 格，则偏移量通常关联这个转动数值。
- n (数组总长度)：循环队列或轨道的总格子数，即取模的基数 (Modulus)。

逻辑拆解：公式中的 $+ n$ 是为了解决 $a < (b \pmod n)$ 时出现的负数情况。这相当于在数轴上向正方向预拨了一整圈，确保分子部分始终为正数，从而让 $\%$ 运算符输出符合预期的物理下标 $[0, n - 1]$ 。

- 题目链接：[跨年赛 div3 春运 \(点击此处\)](#);

代码示例：

```
1 auto solve() -> void {
2     ll d, n, m, t;
3     cin >> d >> n >> m >> t;
4     vector<ll> v(n);
5     for (ll i = 0; i < n; i++) {
6         cin >> v[i];
7     }
8     ll L = m * n;
9     ll l = min(L, t);
10    ll cycle = 0;
11    for (ll i = 0; i < l; i++) {
12        ll count = i / m + 1;
13        ll curr_idx = i % n;
14        ll real_idx = (curr_idx - (count % n) + n) % n;
15        cycle = (cycle + v[real_idx]) % d;
16    }
17    if (t < L) {
18        cout << cycle << endl;
19    } else {
20        ll L_count = t / L;
21        ll res = t % L;
22        ll cycle_total = (L_count % d * cycle % d) % d;
23        ll res_day = 0;
```

```

24     for (ll i = 0; i < res; i++) {
25         ll count = i / m + 1;
26         ll curr_idx = i % n;
27         ll real_idx = (curr_idx - (count % n) + n) % n;
28         res_day = (res_day + v[real_idx]) % d;
29     }
30     cout << (res_day + cycle_total) % d << endl;
31 }
32 }

```

2.8.5 裴蜀定理

知识点详解：裴蜀定理

- 对于任意整数 a, b , 存在整数 x, y 使得:

$$ax + by = \gcd(a, b)$$

- 推广到 n 个数的情况: 对于序列 $A = \{A_1, A_2, \dots, A_n\}$, 满足 $\sum_{i=1}^n A_i X_i > 0$ 的最小正整数解为:

$$S_{\min} = \gcd(|A_1|, |A_2|, \dots, |A_n|)$$

- 题目链接: [P4549 裴蜀定理 \(点击此处\)](#)

代码示例:

```

1  auto gcd(ll a, ll b) -> ll {
2      while (b) {
3          a %= b;
4          swap(a, b);
5      }
6      return a;
7  }
8
9  auto solve() -> void {
10     ll n;
11     if (!(cin >> n)) return;
12     vector<ll> v(n);
13     for (ll i = 0; i < n; i++) {
14         cin >> v[i];
15         if (v[i] < 0) {
16             v[i] = -v[i];
17         }
18     }
19     ll ans = 0;
20     for (ll i = 0; i < n; i++) {
21         if (i == 0) {
22             ans = v[i];

```

```
23     } else {  
24         ans = gcd(ans, v[i]);  
25     }  
26 }  
27 cout << ans << endl;  
28 }
```


3 算法学习篇 (题目主要来自 Luogu, 但也有部分其他网站的题)

在本节中, 我将整理算法的学习题库¹。

3.1 遇到的问题以及学到的方法

3.1.1 使用的 main 函数和准备工作都存在这里了

代码展示:

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3 #define MOD 1000000007
4 constexpr long long N = 1e6 + 10;
5 using namespace std;
6 using ll = long long;
7 using ull = unsigned long long;
8
9
10 auto solve() -> void {
11
12 }
13
14 int main() {
15     ios::sync_with_stdio(false);
16     cin.tie(nullptr);
17     ll t = 1;
18     // cin >> t;
19     while (t--) {
20         solve();
21     }
22     return 0;
23 }
```

3.2 深度优先搜索

3.2.1 P11388 飞跃

- 题目链接: [P11388 飞跃 \(点击此处\)](#); 方法: 当目标在随机范围内取得时, 可以用条件来将满足条件的值限制在一个范围内, 为保证范围内的值都满足, 往往找最大值和最小值;

代码示例:

¹由于算法模板大部分都来自 Luogu, 所以这章中不全是 Luogu 的题

```

1 void solve() {
2     ll n,k;
3     cin>>n>>k;
4     vector<ll> v(n+1);
5     for (ll i=1;i<=n;i++) {
6         cin>>v[i];
7     }
8     ll mmin=v[1];
9     ll mmax=v[1];
10    cout<<"1 ";
11    for (ll i=2;i<=n;i++) {
12        if (v[i]>=mmin-k&&v[i]<=mmax+k) {
13            cout<<"1 ";
14            mmin=min(mmin,v[i]);
15            mmax=max(mmax,v[i]);
16        }else {
17            cout<<"0 ";
18        }
19    }
20 }

```

3.2.2 递归实现排列型枚举 (有顺序)

- 题目链接: [B3623 枚举排列 \(点击此处\)](#);

代码示例:

```

1 ll n,m;
2 ll res[20];
3 bool vis[20];
4 auto dfs(const ll &step) -> void {
5     if (step > m) {
6         for (ll i = 1; i <= m; i++) {
7             cout << res[i] << " " << endl;
8         }
9         cout << endl;
10        return;
11    }
12    for (ll i = 1; i <= n; i++) {
13        if (!vis[i]) {
14            vis[i] = true;
15            res[step] = i;
16            dfs(step + 1);
17            vis[i] = false;
18        }
19    }
20 }
21
22 auto solve() -> void {
23     cin >> n >> m;
24     dfs(1);
25 }

```

3.2.3 递归实现组合型枚举 (无顺序)

- 题目链接: [P1157 组合的输出 \(点击此处\)](#) 解释 solve 中 x 取 0 和 1 的区别: 1、取 0: 表示已经取了 x 个数; 2、取 1: 表示正在取第 x 个数。

代码示例:

```

1 ll n, m, res[20];
2
3 auto dfs(const ll &x, const ll &start) -> void {
4     if (x > m) {
5         //这里的x到底是>m还是==m取决于solve中传的x的值, 1则>,0则==。
6         for (ll i = 1; i <= m; i++) {
7             cout << (i > 1 ? " " : "") << res[i];
8         }
9         cout << endl;
10        return; //无论是什么类型的搜索都必须要有return;
11    }
12
13    for (ll i = start; i <= n; i++) { //因为组合型每次都从当前开始的位置填充,
14        res[x] = i;                //所以不需要bool类型的数组
15        dfs(x + 1, start + 1);
16    }
17 }
18
19 auto solve() -> void {
20     cin >> n >> m;
21     dfs(1, 1);
22 }

```

3.2.4 递归实现指数型枚举

- 题目链接: [\(AcWing\)92 递归实现指数型枚举 \(点击此处\)](#)

代码链接:

```

1 ll n, res[16];
2
3 auto dfs(const ll &x) -> void {
4     if (x > n) {
5         for (ll i = 1; i <= n; i++) {
6             if (res[i] == 1) {
7                 cout << i << " ";
8             }
9         }
10        cout << endl;
11        return;
12    }
13    //不选
14    res[x] = 2;
15    dfs(x + 1);
16
17    //选

```

```
18     res[x] = 1;
19     dfs(x + 1);
20 }
21
22 auto solve() -> void {
23     cin >> n;
24     dfs(1);
25 }
```

3.2.5 指数型变式

- 题目链接: [P2089 烤鸡 \(点击此处\)](#)

代码示例:

```
1 ll res[15], n, cnt = 0;
2 ll a[59055][15];
3
4 auto dfs(const ll &x, const ll &sum) -> void {
5     if (sum > n) return;
6     if (x > 10) {
7         if (sum == n) {
8             cnt++;
9             for (ll i = 1; i <= 10; i++) {
10                 a[cnt][i] = res[i];
11             }
12         }
13         return;
14     }
15     for (ll i = 1; i <= 3; i++) {
16         res[x] = i;
17         dfs(x + 1, sum + i);
18     }
19 }
20
21 auto solve() -> void {
22     cin >> n;
23     cnt = 0;
24     dfs(1, 0);
25     cout << cnt << endl;
26     for (ll i = 1; i <= cnt; i++) {
27         for (ll j = 1; j <= 10; j++) {
28             cout << a[i][j] << " ";
29         }
30         cout << endl;
31     }
32 }
```

3.2.6 排列型变式

- 题目链接: [P1088 火星人 \(点击此处\)](#)

代码链接:

```

1 //法一: 库函数实现全排列
2 auto solve()->void{
3     ll n,m;
4     vector<ll> a(10005);
5     for(ll i=0;i<n;i++){
6         cin>>a[i];
7     }
8     while(m--){
9         next_permutation(a.begin(),a.end());
10    }
11    for(ll i=0;i<n;i++){
12        cout<<(i>1?" ":"")<<a[i];
13    }
14    cout<<endl;
15 }
16
17 //法二: DFS
18 ll n, m, res[10005];
19 ll a[10005], cnt = 0;
20 bool vis[10005];
21
22 auto dfs(const ll &x) {
23     if (x > n) {
24         cnt++;
25         if (cnt == m + 1) {
26             for (ll i = 1; i <= n; i++) {
27                 cout << (i > 1 ? " " : "") << res[i];
28             }
29             cout << endl;
30             exit(0);
31         }
32         return; //即使上面有exit这一步也一定不能少。
33     }
34     for (ll i = 1; i <= n; i++) {
35         //强制让dfs从当前数字进行排序
36         if(!cnt){ //当cnt=0时就让排列强制从以a数组开始。这样第一次就是给定数组了
37             i=a[x];
38         }
39         if (!vis[i]) {
40             vis[i] = true;
41             res[x] = i;
42             dfs(x + 1);
43             vis[i] = false;
44         }
45     }
46 }
47
48 auto solve() -> void {
49     cin >> n >> m;
50     for (ll i = 1; i <= n; i++) {
51         cin >> a[i];
52     }
53     dfs(1);
54 }

```

3.2.7 连通块问题

- 题目链接: [P1596Lake Counting S\(点击此处\)](#)

代码示例:

```

1  ll dx[8] = {-1, 1, 0, 0, -1, -1, 1, 1};
2  ll dy[8] = {0, 0, -1, 1, -1, 1, -1, 1};
3  ll g[105][105], n, m;
4
5  auto dfs(const ll &x, const ll &y) -> void {
6      //把符合条件的块全都标记成0
7      g[x][y] = 0;
8      for (ll i = 0; i < 8; i++) {
9          ll xx = x + dx[i];
10         ll yy = y + dy[i];
11         if (xx < 1 || yy < 1 || xx > n || yy > m || g[xx][yy] == 0) {
12             continue;
13         }
14         dfs(xx, yy);
15     }
16 }
17
18 auto bfs(const ll &x, const ll &y) -> void {
19     queue<pair<ll, ll> > q;
20     g[x][y] = 0;
21     q.emplace(x, y);
22     while (!q.empty()) {
23         const auto &[fis, snd] = q.front();
24         q.pop();
25         for (ll i = 0; i < 8; i++) {
26             ll xx = fis + dx[i];
27             ll yy = snd + dy[i];
28             if (xx < 1 || yy < 1 || xx > n || yy > m || g[xx][yy] == 0) {
29                 continue;
30             }
31             q.emplace(xx, yy);
32             g[xx][yy] = 0;
33         }
34     }
35 }
36
37 auto solve() -> void {
38     ll cnt = 0;
39     cin >> n >> m;
40     map<char, ll> mp;
41     mp['W'] = 1;
42     mp['.'] = 0;
43     for (ll i = 1; i <= n; i++) {
44         for (ll j = 1; j <= m; j++) {
45             char ch;
46             cin >> ch;
47             g[i][j] = mp[ch];
48         }
49     }
50     for (ll i = 1; i <= n; i++) {
51         for (ll j = 1; j <= m; j++) {
52             //因为有标记0的步骤, 所以接下来只要不是0的就是一个块的开始
53             if (g[i][j] > 0) {
54                 cnt++;
55                 //dfs(i, j);
56                 bfs(i, j);
57             }
58         }
59     }
60     cout << cnt << endl;

```

```
61 }
```

- 题目链接: [P1683 入门 \(点击此处\)](#)

代码示例:

```
1 ll w, h;
2 ll v[25][25], ans=0;
3 ll dx[4] = {-1, 1, 0, 0};
4 ll dy[4] = {0, 0, -1, 1};
5
6 auto dfs(const ll &x, const ll &y) -> void {
7     v[x][y] = 0;
8     ans++;
9     for (ll i = 0; i < 4; i++) {
10         ll nx = x + dx[i];
11         ll ny = y + dy[i];
12         if (nx > w || ny > h || nx < 1 || ny < 1 || v[nx][ny]==0) {
13             continue;
14         }
15         dfs(nx, ny);
16     }
17 }
18
19 auto solve() -> void {
20     cin >> w >> h;
21     ans = 0;
22     ll a, b;
23     map<char, ll> mp;
24     mp['.'] = 2;
25     mp['#'] = 0;
26     mp['@'] = 2;
27     for (ll y = 1; y <= h; y++) {
28         for (ll x = 1; x <= w; x++) {
29             char ch;
30             cin >> ch;
31             if (ch=='@') {
32                 a=x;
33                 b=y;
34             }
35             v[x][y] = mp[ch];
36         }
37     }
38     dfs(a, b);
39     cout << ans << endl;
40 }
```

3.2.8 P1644 跳马

- 题目链接: [P1644 跳马 \(点击此处\)](#)

代码示例:

```
1 ll n, m, ans = 0;
```

```

2  ll dx[4] = {-2, -1, 1, 2};
3  ll dy[4] = {1, 2, 2, 1};
4
5  auto dfs(const ll &x, const ll &y) -> void {
6      if (x == n && y == m) {
7          ans++;
8          return;
9      }
10     for (ll i = 0; i < 4; i++) {
11         ll nx = x + dx[i];
12         ll ny = y + dy[i];
13         if (nx < 0 || ny < 0 || nx > n || ny > m) {
14             continue;
15         }
16         dfs(nx, ny);
17     }
18 }
19
20 auto solve() -> void {
21     cin >> n >> m;
22     ans = 0;
23     dfs(0, 0);
24     cout << ans << endl;
25 }

```

3.2.9 P2036 放调料

- 题目链接: [P2036PERKET\(点击此处\)](#)

代码示例:

```

1  //对下面的代码进行优化
2  /*
3  // sd: 当前总酸度, kd: 当前总苦度, count: 已选调料数量
4  auto dfs(ll x, ll sd, ll kd, ll count) -> void {
5      if (x > n) {
6          if (count > 0) { // 只有选了调料才更新最小值
7              mmin = min(mmin, abs(sd - kd));
8          }
9          return;
10     }
11
12     // 1. 选当前这件调料
13     dfs(x + 1, sd * a[x][0], kd + a[x][1], count + 1);
14
15     // 2. 不选当前这件调料
16     dfs(x + 1, sd, kd, count);
17 }
18
19 // 在 solve 中调用
20 dfs(1, 1, 0, 0); // 初始酸度为1, 苦度为0, 数量为0
21 */
22 //优化前
23 ll n, a[11][2], mmin = INT_MAX;
24 ll res[11];
25
26 auto dfs(const ll &x) -> void {
27     ll sd=1,kd=0; //一定要注意sd是要做乘积的, 不能初始化为0

```



```

28     bool flag=false; //一定要注意组合型里面有一种为空的情况, 但是题目要求至少选一种。
29     if (x > n) {
30         for (ll i = 1; i <= n; i++) {
31             if (res[i] == 1) { //满足至少一种
32                 sd += a[i][0];
33                 kd += a[i][1];
34                 flag=true;
35             }
36         }
37         if (flag) {
38             mmin = min(mmin, abs(sd - kd));
39         }
40         return;
41     }
42
43     //不选
44     res[x] = 2;
45     dfs(x + 1);
46     //选
47     res[x] = 1;
48     dfs(x + 1);
49 }
50
51 auto solve() -> void {
52     cin >> n;
53     mmin = INT_MAX;
54     for (ll i = 1; i <= n; i++) {
55         cin >> a[i][0] >> a[i][1];
56     }
57     dfs(1);
58     cout << mmin << endl;
59 }

```

3.2.10 P1149 火柴棒等式

- 题目链接: [P1149 火柴棒等式 \(点击此处\)](#)

代码示例:

```

1  ll n, res[5], cnt = 0;
2  ll to[10] = {6, 2, 5, 5, 4, 5, 6, 3, 7, 6};
3
4  auto total(ll x) -> ll {
5      ll ans = 0;
6      if (x == 0) return 6;
7      while (x) {
8          ans += to[x % 10];
9          x /= 10;
10     }
11     return ans;
12 }
13
14 auto dfs(const ll &x) -> void {
15     if (x > 2) {
16         ll c = res[1] + res[2];
17         if (total(res[1]) + total(res[2]) + total(c) == n - 4) {
18             cnt++;
19         }

```

```

20     return;
21 }
22
23 for (ll i = 0; i <= N; i++) {
24     res[x] = i;
25     dfs(x + 1);
26 }
27 }
28
29 auto solve() -> void {
30     cin >> n;
31     cnt = 0;
32     dfs(1);
33     cout << cnt << endl;
34 }

```

3.2.11 P1036 选数

- 题目链接: [P1036 选数 \(点击此处\)](#)

代码示例:

```

1 ll n, m, ans = 0;
2 ll num[30];
3 //素数判断的优化, 一定要记牢会写, 可以大大提高程序效率
4 bool isprime(const ll &y) {
5     if (y < 2) return false;
6     if (y == 2 || y == 3) return true;
7     if (y % 2 == 0 || y % 3 == 0) return false;
8     for (ll i = 5; i * i <= y; i += 6) {
9         if (y % i == 0 || y % (i + 2) == 0) return false;
10    }
11    return true;
12 }
13
14 auto dfs(const ll &x, const ll &start, const ll &cnt) -> void {
15     //剪枝
16     if (x + n - start + 1 < m) {
17         return;
18     }
19
20     if (x == m) {
21         if (isprime(cnt)) {
22             ans++;
23         }
24         return;
25     }
26     for (ll i = start; i <= n; i++) {
27         dfs(x + 1, i + 1, cnt + num[i]);
28     }
29 }
30
31 auto solve() -> void {
32     ans = 0;
33     cin >> n >> m;
34     for (ll i = 1; i <= n; i++) {
35         cin >> num[i];
36     }

```

```

37     dfs(0, 1, 0);
38     cout << ans << endl;
39 }

```

3.2.12 P1135 选和不选类 (多方案选择类)

- 题目链接:[P1135 奇怪的电梯 \(点击此处\)](#) **注意:** 记忆化的技巧很常见一定要知道!

代码示例:

```

1  ll n, a, b;
2  ll v[202], ans[205];
3
4  auto dfs(const ll &x, const ll &step) -> void {
5      //ans作为记忆化数组, 当步数大于当前最小步就没有继续下去的必要了
6      if (step >= ans[x]) return;
7      ans[x] = step;
8      if (x == b) {
9          return;
10     }
11     //上
12     if (x + v[x] <= n) dfs(x + v[x], step + 1);
13     //下
14     if (x - v[x] >= 1) dfs(x - v[x], step + 1);
15 }
16
17 auto solve() -> void {
18     cin >> n >> a >> b;
19     for (ll i = 1; i <= n; i++) {
20         cin >> v[i];
21         ans[i] = INT_MAX;
22     }
23     dfs(a, 0);
24     if (ans[b] == INT_MAX) {
25         cout << -1 << endl;
26         return;
27     }
28     cout << ans[b] << endl;
29 }

```

3.2.13 P1025 数的划分

- 题目链接:[p1025 数的划分 \(点击此处\)](#)

代码示例:

```

1  ll n, k;
2  ll ans = 0;
3
4  auto dfs(const ll &x, const ll &step, const ll &sum) -> void {
5      if (x == k) { //若x=1, 这里也要相应的改成x>k.原因前面讲过了。
6          if (sum == n) {

```

```

7         ans++;
8     }
9     return;
10 }
11 //若下面的x从1开始, 那么剪枝会发生相应的变化应是: sum+i*(k-(x-1))<=n
12 //因为当x取1时, 此时的x表示这在选的数字, 实际上还没选, 所以要-1
13 for (ll i = step; sum + i * (k - x) <= n; i++) {
14     //这里的i不加1的原因是题目是可以取之前的数字的组合问题, 若为i+1则只能往后取。
15     //而从i开始就是说这次取1下次还可以取1.
16     dfs(x + 1, i, sum + i);
17 }
18 }
19
20 auto solve() -> void {
21     cin >> n >> k;
22     ans = 0;
23     dfs(0, 1, 0);
24     cout << ans << endl;
25 }

```

3.3 二分答案

方法: 二分答案就是找到要求的值为 x , 要满足的最小或者最大条件为 y

3.3.1 AcWing789 二分模板 (开区间)(点击此处)

- 题目链接: [AcWing789 二分模板 \(开区间\)](#)

代码示例:

```

1 auto solve() -> void {
2     ll n, q;
3     cin >> n >> q;
4     vector<ll> v(n);
5     for (ll i = 0; i < n; i++) {
6         cin >> v[i];
7     }
8     while (q--) {
9         ll x;
10        cin >> x;
11        ll l = -1, r = n;
12        while (l + 1 < r) {
13            ll mid = l + ((r - l) >> 1);
14            if (v[mid] >= x) {
15                r = mid;
16            } else {
17                l = mid;
18            }
19        }
20        if (r == n || v[r] != x) {
21            cout << -1 << " " << -1 << endl;
22            continue;
23        }
24        ll strat = r;
25        l = -1, r = n;
26        while (l + 1 < r) {

```

```

27         ll mid = l + ((r - l) >> 1);
28         if (v[mid] <= x) {
29             l = mid;
30         } else {
31             r = mid;
32         }
33     }
34     ll end = l;
35     cout << strat << " " << end << endl;
36 }
37 }

```

3.3.2 P2678 跳石头

- 题目链接:[P2678 跳石头 \(点击此处\)](#); 代码示例:

```

1 bool check(ll x) {
2     ll ans = 0, last = 0;
3     for (ll i = 1; i <= n+1; i++) {
4         if (v[i] - v[last] < x) {
5             ans++;
6         } else {
7             last = i;
8         }
9     }
10    return ans <= m;
11 }
12
13 void solve() {
14     cin >> l >> n >> m;
15     v[0] = 0;
16     for (ll i = 1; i <= n; i++) {
17         cin >> v[i];
18     }
19     v[n + 1] = l;
20     ll left = 0, right = l+1;
21     while (left + 1 < right) {
22         ll mid = left + ((right - left) >> 1);
23         if (check(mid)) {
24             left = mid;
25         } else {
26             right = mid;
27         }
28     }
29     cout << left << endl;
30 }

```

注意：本题可作为二分答案模板，本题方法也要理解；

3.3.3 P1824 进击的奶牛

- 题目链接:[P1824 进击的奶牛 \(点击此处\)](#);**注意：**这题是上一题的变式题，但要做转换；

代码示例:

```

1 ll n,m;
2 bool check(const vector<ll> a,ll x) {
3     ll last=0,y=0;
4     for (ll i=1;i<n;i++) {
5         if (a[i]-a[last]<x) {
6             y++;
7         }else {
8             last=i;
9         }
10    }
11    return y<=n-m;
12 }
13
14 void solve() {
15     cin >> n >> m;
16     vector<ll> v(n + 1), a(n);
17     for (ll i = 1; i <= n; i++) {
18         cin >> v[i];
19     }
20     sort(v.begin(), v.end());
21     ll mmax = 0;
22     a[0]=0;
23     for (ll i = 1; i < n; i++) {
24         a[i] = v[i + 1] - v[1];
25         mmax = max(mmax, a[i]);
26     }
27     ll l = 0, r = mmax + 1;
28     while (l + 1 < r) {
29         ll mid = l + ((r - l) >> 1);
30         if (check(a,mid)) {
31             l = mid;
32         } else {
33             r = mid;
34         }
35     }
36     cout << l << endl;
37 }

```

3.3.4 B3629 吃冰棍

- 题目链接:[B3629 吃冰棍 \(点击此处\)](#);

代码示例:

```

1 ll exp(ll n) {
2     ll ans = n, exchage;
3     while (n >= 3) {
4         exchage = n / 3;
5         ans += exchage;
6         n = n % 3 + exchage;
7     }
8     return ans;
9 }
10

```

```

11 int main() {
12     ios_base::sync_with_stdio(false);
13     cin.tie(nullptr);
14     ll n, ans = 0;
15     cin >> n;
16     ll l = 0, r = n;
17     while (l + 1 < r) {
18         ll mid = l + ((r - l) >> 1);
19         if (exp(mid) >= n) {
20             ans = mid;
21             r = mid;
22         } else {
23             l = mid;
24         }
25     }
26     cout << ans << endl;
27     return 0;
28 }

```

3.3.5 P1918 保龄球 (二分法)

- 题目链接: [P1918 保龄球 \(点击此处\)](#); 总结: 二分思路因为最后要的是下标, 所以一开始可以用 pair 来存一下下标; [法二在哈希表那里](#)

代码示例:

```

1 int main() {
2     ios_base::sync_with_stdio(false);
3     cin.tie(nullptr);
4     ll n;
5     cin >> n;
6     vector<ll> v(n + 1);
7     for (ll i = 1; i <= n; i++) {
8         cin >> v[i];
9     }
10    vector<pair<ll, ll> > a(n + 1);
11    for (ll i = 1; i <= n; i++) {
12        a[i] = {v[i], i};
13    }
14    sort(a.begin()+1, a.begin()+n+1);
15    ll q;
16    cin >> q;
17    while (q--) {
18        ll t;
19        cin >> t;
20        ll l = 0, r = n + 1;
21        while (l + 1 < r) {
22            ll mid = l + ((r - l) >> 1);
23            if (a[mid].first <= t) {
24                l = mid;
25            } else {
26                r = mid;
27            }
28        }
29        if (l <= n && l >= 1 && a[l].first == t) {
30            cout << a[l].second << endl;
31        } else {

```

```

32     cout << 0 << endl;
33 }
34 }
35 return 0;
36 }

```

3.3.6 提高组二分答案 (P1083)

- 题目链接: [P1083 借教室 \(点击此处\)](#); 总结: 本题的二分比较隐晦不好看出, 需要通过大量的题去训练。要搞清楚分什么, 比如本题就是分订单编号。二分答案一般都是没有排序步骤的, 所以一般情况下, 本身就具有顺序的那个量就是要二分的量。比如本题的订单是按顺序执行的所以是顺序量。

代码示例:

```

1  ll s[N], d[N], t[N], r[N], b[N];
2  ll n, m;
3
4  bool check(ll x) {
5      for (ll i = 1; i <= n; i++) b[i] = r[i];
6      for (ll i = 1; i <= x; i++) {
7          b[s[i]] -= d[i];
8          b[t[i] + 1] += d[i];
9      }
10     ll sum = 0;
11     for (ll i = 1; i <= n; i++) {
12         sum += b[i];
13         if (sum < 0) {
14             return true;
15         }
16     }
17     return false;
18 }
19
20 void solve() {
21     cin >> n >> m;
22     for (ll i = 1; i <= n; i++) {
23         cin >> r[i];
24     }
25     //这里需要特别注意: 在数组本身进行差分是一定要倒序差分, 不然会
26     //影响下一项, 也可以重新开一个数组来存放差分, 就不需要倒序。
27     for (ll i = n; i >= 1; i--) {
28         r[i] -= r[i - 1];
29     }
30     for (ll i = 1; i <= m; i++) {
31         cin >> d[i] >> s[i] >> t[i];
32     }
33     ll l = 0, right = m + 1;
34     while (l + 1 < right) {
35         ll mid = l + ((right - l) >> 1);
36         if (check(mid)) {
37             right = mid;
38         } else {
39             l = mid;
40         }

```



```
41     }
42     if (check(right)) {
43         cout << -1 << endl;
44         cout << right << endl;
45     } else {
46         cout << 0 << endl;
47     }
48 }
```

再次强调：在数组本身做差分时一定要倒序!!!

3.3.7 P1577 切绳子

- 题目链接：P1577 切绳子 ([点击此处](#))；总结：因为题目给的绳子长度都是有两位小数，所以可以将绳子长度先乘 100 然后当成整数二分会方便很多；

代码示例：

```
1  float v[200005];
2  ll a[200005];
3  ll n, k;
4
5  bool check(ll x) {
6      ll sum = 0;
7      for (ll i = 0; i < n; i++) {
8          sum += a[i]/x;
9      }
10     return sum >= k;
11 }
12
13 void solve() {
14     cin >> n >> k;
15     ll mmax = 0;
16     for (ll i = 0; i < n; i++) {
17         cin >> v[i];
18     }
19     //注意：这里不能直接在v数组上操作因为类型不同
20     for (ll i=0;i<n;i++) {
21         a[i]=(ll)(v[i]*100);
22         mmax=max(mmax,a[i]);
23     }
24     ll l = 0, r = mmax + 1;
25     while (l + 1 < r) {
26         ll mid = l + ((r - l) >> 1);
27         if (check(mid)) {
28             l = mid;
29         } else {
30             r = mid;
31         }
32     }
33     float z = l / 100.0;
34     cout << fixed << setprecision(2) << z << endl;
35 }
```

3.3.8 二分进阶 (newcodeF)

- 题目链接: [扑克牌 F\(点击此处\)](#); 总结: 像上一题说的二分答案一般会让排序, 所以一定有一个单调的量。本题就是组合的套数;

代码示例:

```

1 ll c[60], m, n;
2
3 auto check(const ll &k) -> bool {
4     ll cnt_joker = 0;
5     for (ll i = 1; i <= n; i++) {
6         if (c[i] < k) {
7             cnt_joker += k - c[i];
8         }
9     }
10    //这里特别注意, 同一套牌里不能出现两个joker, 所以cnt_joker还要<=k;
11    return cnt_joker <= m && cnt_joker <= k;
12 }
13
14 auto solve() -> void {
15     cin >> n >> m;
16     ll ans_max = 0;
17     for (ll i = 1; i <= n; i++) {
18         cin >> c[i];
19         ans_max = max(ans_max, c[i]);
20     }
21     ll l = 0, r = ans_max + m;
22     while (l + 1 < r) {
23         if (ll mid = l + ((r - l) >> 1); check(mid)) {
24             l = mid;
25         } else {
26             r = mid;
27         }
28     }
29     cout << l << endl;
30 }

```

3.3.9 聪明的质检员

- 题目链接: [P1314 聪明的质检员 \(点击此处\)](#) 总结: 这题解锁了前缀和的新用法, 所以当数组前一个等于后一个时, 两处的前缀和是不变的

代码示例:

```

1 ll n, m, s;
2 ll w[N], v[N], a[N];
3 ll l1[N], r1[N], cnt[N], sumv[N];
4
5 auto check(const ll wx) -> ll {
6     ll sum = 0;
7     for (ll i = 1; i <= n; i++) {

```

```

8         if (w[i] >= wx) {
9             cnt[i] = cnt[i - 1] + 1;
10            sumv[i] = sumv[i - 1] + v[i];
11        } else {
12            cnt[i] = cnt[i - 1];
13            sumv[i] = sumv[i - 1];
14        }
15    }
16    for (ll i = 1; i <= m; i++) {
17        sum += (cnt[r1[i]] - cnt[l1[i] - 1]) * (sumv[r1[i]] - sumv[l1[i] - 1]);
18    }
19    return sum;
20 }
21
22 auto solve() -> void {
23     cin >> n >> m >> s;
24     ll mmax = 0, mmin = INT_MAX;
25     for (ll i = 1; i <= n; i++) {
26         cin >> w[i] >> v[i];
27         mmax = max(mmax, w[i]);
28         mmin = min(mmin, w[i]);
29     }
30     for (ll i = 1; i <= n; i++) {
31         a[i] = a[i - 1] + v[i];
32     }
33     for (ll i = 1; i <= m; i++) {
34         cin >> l1[i] >> r1[i];
35     }
36     ll l = mmin - 1, r = mmax + 2;
37     ll ans = 1e18; //这里的ans一定要取大一点, 因为abs(y-s)有可能会非常小。
38     while (l + 1 < r) {
39         ll mid = l + ((r - l) >> 1);
40         ll y = check(mid);
41         ans = min(ans, abs(y - s));
42         if (y < s) {
43             r = mid;
44         } else if (y > s) {
45             l = mid;
46         } else {
47             ans = 0;
48             break;
49         }
50     }
51     cout << ans << endl;
52 }

```

3.3.10 P5119 奶牛上车

- 题目链接: [P5119 奶牛上车 \(点击此处\)](#) **注意:** 这里要注意一辆车不必坐满了再走。

代码示例:

```

1 ll n, m, c;
2 vector<ll> v;
3
4 auto check(const ll x) -> bool {
5     ll ans = 1;
6     ll start_id = 0;

```

```

7     for (ll i = 1; i < n; i++) {
8         if (v[i] - v[start_id] > x || i - start_id + 1 > c) {
9             ans++;
10            start_id = i;
11        }
12    }
13    return ans <= m;
14 }
15
16 auto solve() -> void {
17     cin >> n >> m >> c;
18     for (ll i = 0; i < n; i++) {
19         ll num;
20         cin >> num;
21         v.emplace_back(num);
22     }
23     sort(v.begin(), v.end());
24     ll l = 0, r = v[n - 1] - v[0] + 1;
25     while (l + 1 < r) {
26         ll mid = l + ((r - l) >> 1);
27         if (check(mid)) {
28             r = mid;
29         } else {
30             l = mid;
31         }
32     }
33     cout << r << endl;
34 }

```

总结：在解决最大的最小值或最小的最大值时，往往要求的東西为二分对象，同时也是 check 函数里面的比较对象；

3.3.11 P2650 弹幕考察

- 题目链接: [pp2650 弹幕考察 \(点击此处\)](#)

代码示例:

```

1 auto solve() -> void {
2     ll n, m;
3     cin >> n >> m;
4     ll l[n], r[n], le;
5     for (ll i = 0; i < n; i++) {
6         cin >> l[i] >> le;
7         r[i] = l[i] + le;
8     }
9     sort(l, l + n);
10    sort(r, r + n);
11    while (m--) {
12        ll a, b;
13        cin >> a >> b;
14        ll c = a + b;
15        ll countl = lower_bound(l, l + n, c) - l;
16        ll countr = upper_bound(r, r + n, a) - r;
17        cout << countl - countr << endl;
18    }

```

```
19 }
```

3.3.12 2026 天梯选拔 2-1

- 题目链接: [2-1 魔法温室 \(点击此处\)](#);

总结: 本题为二分和差分的综合应用, 对于本题要主要掌握差分的用法!

代码示例:

```
1 ll n, m, w;
2 ll v[N];
3
4 auto check(ll x) -> bool {
5     vector<ll> diff(n + 1, 0);
6     ll need_total = 0;
7     ll total_cnt = 0;
8     for (ll i = 0; i < n; i++) {
9         need_total += diff[i]; //模拟了对差分求前缀和的过程对应了13行的更新
10        ll total_h = v[i] + need_total;
11        if (total_h < x) {
12            ll need = x - total_h;
13            total_cnt += need;
14            if (total_cnt > m) {
15                return false;
16            }
17            need_total += need;
18            if (i + w < n) {
19                diff[i + w] -= need;
20            }
21        }
22    }
23    return total_cnt <= m;
24 }
25
26 auto solve() -> void {
27     cin >> n >> m >> w;
28     ll mmax = -1e18, mmin = 1e18;
29     for (ll i = 0; i < n; i++) {
30         cin >> v[i];
31         mmax = max(mmax, v[i]);
32         mmin = min(mmin, v[i]);
33     }
34     ll left = mmin - 1, right = mmax + m;
35     while (left + 1 < right) {
36         ll mid = left + ((right - left) >> 1);
37         if (check(mid)) {
38             left = mid;
39         } else {
40             right = mid;
41         }
42     }
43     cout << left << endl;
44 }
```

3.4 分数规划

分数规划 (Fractional Programming) 指的是求解形如

$$\max_{x \in S} \frac{f(x)}{g(x)},$$

的优化问题, 其中 $f(x)$ 和 $g(x)$ 为实函数且满足 $g(x) > 0$ 。其核心思想是将分式目标转换为等价的参数化形式, 例如通过 Dinkelbach 方法, 将其转化为一系列

$$\max_{x \in S} f(x) - \lambda g(x)$$

的子问题来求解。

3.4.1 P1570KC 咖啡

- 题目链接: [P1570KC 咖啡 \(点击此处\)](#); **注意:** 此题将作为分数规划的模板;

代码示例:

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4 using ll = long long;
5 using ull = unsigned long long;
6 const ll N = 1010;
7 ll n, m;
8 double v[N], b[N], c[N];
9
10 bool check(double x) {
11     double s=0;
12     for (ll i=1;i<=n;i++) {
13         b[i]=v[i]-x*c[i];
14     }
15     sort(b+1,b+n+1,greater<double>());
16     for (ll i=1;i<=m;i++) {
17         s+=b[i];
18     }
19     return s>=0;
20 }
21
22 double find() {
23     double l=0,r=1e6;
24     while (r-l>1e-5) {
25         double mid=l+(r-l)/2;
26         if (check(mid)) {
27             l=mid;
28         }else {
29             r=mid;
30         }
31     }
```

```
31     }
32     return 1;
33 }
34
35 void solve() {
36     cin>>n>>m;
37     for (ll i=1;i<=n;i++) {
38         cin>>v[i];
39     }
40     for (ll i=1;i<=n;i++) {
41         cin>>c[i];
42     }
43     printf("%.3lf\n",find());
44 }
45
46 int main() {
47     // ios::sync_with_stdio(false);
48     // cin.tie(nullptr);
49     ll t = 1;
50     //cin >> t;
51     while (t-->0) {
52         solve();
53     }
54     return 0;
55 }
```

3.5 哈希表

哈希表 (Hash Table): 一种利用哈希函数将键映射到数组下标, 从而实现近似 $O(1)$ 时间的查找、插入和删除的数据结构。

用途: 快速判断元素是否存在、统计频次、建立键值映射、实现去重等。用途: 快速判断元素是否存在、统计频次、建立键值映射、实现去重等。

3.5.1 A+B

- 题目链接: [A+B\(点击此处\)](#);

代码示例:

```
1 class Solution {
2     public:
3     vector<int> twoSum(vector<int>& nums, int target) {
4         map<int,int> mp;
5         vector<int> b;
6         for(int i=0;i<nums.size();i++){
7             if(mp.count(target-nums[i])){
8                 b.push_back(mp[target-nums[i]]);
9                 b.push_back(i);
10                return b;
11            }
12            mp[nums[i]]=i;
```

```
13     }
14     return b;
15 };
16 };
```

3.5.2 A-B

- 题目链接: [A-B 数对 \(点击此处\)](#);

代码示例:

```
1 void solve(){
2     ll n,c;
3     cin>>n>>c;
4     vector<ll> v(n);
5     unordered_map<ll,ll> mp;
6     for(ll i=0;i<n;i++){
7         cin>>v[i];
8         mp[v[i]]++;
9         v[i]-=c;
10    }
11    ll ans=0;
12    for(ll i=0;i<n;i++){
13        ans+=mp[v[i]];
14    }
15    cout<<ans<<endl;
16 }
```

3.5.3 P1918 保龄球 (哈希表)

- 题目链接: [P1918 保龄球 \(点击此处\)](#); 总结: 哈希表的查找函数 count; 法一在二分那里

代码示例:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int main() {
6     ll n;
7     cin >> n;
8     vector<ll> v(n + 1);
9     unordered_map<ll, ll> mp;
10    for (int i = 1; i <= n; i++) {
11        cin >> v[i];
12        mp[v[i]] = i;
13    }
14    ll q;
15    cin >> q;
16    while (q--) {
17        ll t;
18        cin >> t;
```



```

19     if (mp.count(t)) {
20         cout << mp[t] << endl;
21     } else {
22         cout << "0" << endl;
23     }
24 }
25 return 0;
26 }

```

3.5.4 哈希表解决数对问题

- 题目链接: [优质数对 \(点击此处\)](#); 总结: 我们要统计 (i, j) 对, 使得:

1. $1 \leq i < j \leq N$
2. $A[i] = B[j]$
3. $A[j] = B[i]$

观察条件 2 和 3, 可以重写为:

$(A[i], B[i])$ 和 $(B[j], A[j])$ 匹配

也就是说, 如果我们将每个位置的 $(B[i], A[i])$ 对统计出现次数, 那么在遍历 i 时, $(A[i], B[i])$ 出现在后面多少次, 就有多少个优质数对。

代码示例:

```

1 void solve() {
2     ll n;
3     cin >> n;
4     vector<ll> a(n);
5     vector<ll> b(n);
6     for (ll i = 0; i < n; i++) cin >> a[i];
7     for (ll i = 0; i < n; i++) cin >> b[i];
8     map<pair<ll, ll>, ll> mp;
9     ll ans = 0;
10    for (ll j = n - 1; j >= 0; j--) {
11        ans += mp[{a[j], b[j]}];
12        mp[{b[j], a[j]}]++;
13    }
14    cout << ans << endl;
15 }

```

注意: 这里的 j 必须从右往左算因为要保证 $i < j$ 的情况下使 $(A[i], B[i]) = (B[j], A[j])$;

- 题目链接:[DNA 序列编辑 \(点击此处\)](#);总结:DNA 序列要满足 $\text{cnt}(A)=\text{cnt}(T), \text{cnt}(C)=\text{cnt}(G)$; 即 $(\text{cnt}(A), \text{cnt}(C))=(\text{cnt}(T), \text{cnt}(G))$; 如下表所示: 当 (d_1, d_2) 数对相等时就会产生一个符合条件的子串。

位置	字符	$d_1 = A - T$	$d_2 = C - G$	(d_1, d_2)
0 (初始)	-	0	0	(0, 0)
1	A	1	0	(1, 0)
2	G	1	-1	(1, -1)
3	C	1	0	(1, 0)
4	T	0	0	(0, 0)

表 1: 例子 1: 字符串"AGCT" 的前缀差分 (d_1, d_2)

代码示例:

```

1 void solve(){
2     ll n;
3     string s;
4     cin>>n>>s;
5     map<pair<ll,ll>,ll> mp;
6     ll d1=0,d2=0,ans=0;
7     for(ll i=0;i<n;i++){
8         char c=s[i];
9         if(c=='A')d1++;
10        else if(c=='T')d1--;
11        else if(c=='C')d2++;
12        else d2--;
13        ans+=mp[{d1,d2}];
14        mp[{d1,d2}]++;
15    }
16    cout<<ans<<endl;
17 }
```

3.5.5 哈希数组 (P3879)

- 题目链接: [P3879 阅读理解 \(点击此处\)](#);

代码示例:

```

1 void solve() {
2     ll n;
3     cin >> n;
4     unordered_map<string, ll> mp[1001]; //定义哈希数组
5     for(ll i = 1; i <= n; ++i) {
6         ll len;
7         cin >> len;
8         while(len--) {
9             string s;
10            cin >> s;
11            mp[i][s]++; //统计第i个哈希表的s单词个数
12        }
13    }
```

```

13     }
14     ll q;
15     cin >> q;
16     while(q--){
17         string s1;
18         cin >> s1;
19         for(ll i = 1; i <= n; i++) {
20             if(mp[i].count(s1)) {
21                 cout << i << " ";
22             }
23         }
24         cout << endl;
25     }
26 }

```

3.6 双指针

3.6.1 盛最多水的容器 (LeetCode)

- 题目链接: [盛最多水的容器 \(点击此处\)](#);

代码示例:

```

1 class Solution {
2     public:
3     int maxArea(vector<int>& height) {
4         int n=height.size();
5         int l=0,r=n-1;
6         int res=0,ans=0;
7         while(l<r){
8             ans=min(height[l],height[r])*(r-l);
9             res=max(res,ans);
10            if(height[l]<height[r]){
11                l++;
12            }else{
13                r--;
14            }
15        }
16        return res;
17    }
18 };

```

3.6.2 接雨水 (LeetCode)

- 题目链接: [接雨水 \(点击此处\)](#); 总结: 需要维护左右两边的最大值;

代码示例:

```

1 class Solution {
2     public:

```

```
3   int trap(vector<int>& h) {
4       int n=h.size();
5       int l=0,r=n-1;
6       int lmax=h[l],rmax=h[r],ans=0;
7       while(l<r){
8           if(h[l]<=h[r]){
9               ans+=lmax-h[l];
10              l++;
11              lmax=max(lmax,h[l]);
12          }else{
13              ans+=rmax-h[r];
14              r--;
15              rmax=max(rmax,h[r]);
16          }
17      }
18      return ans;
19  }
20 };
```

3.6.3 P1147 连续自然数之和

- 题目链接: [P1147 连续自然数之和 \(点击此处\)](#);

代码示例:

```
1 void solve() {
2     ll m;
3     cin>>m;
4     ll l=1,r=1,sum=1;
5     while (l<=m/2) {
6         if (sum<m) {
7             r++;
8             sum+=r;
9         }
10        if (sum>=m) {
11            if (sum==m) {
12                cout<<l<<" "<<r<<endl;
13            }
14            sum-=l;
15            l++;
16        }
17    }
18 }
```

3.6.4 P1638 逛画展

- 题目链接: [P1638 逛画展 \(点击此处\)](#);

代码示例:

```
1 ll paint_type[2005];
2
```

```

3 auto solve() -> void {
4     ll n, m;
5     cin >> n >> m;
6     vector<ll> v(n + 1);
7     for (ll i = 1; i <= n; i++) {
8         cin >> v[i];
9     }
10    ll l = 1, a = 0, b = 0;
11    ll mmin = INT_MAX;
12    ll paint_nums = 0;
13    for (ll r = 1; r <= n; r++) {
14        if (paint_type[v[r]] == 0) {
15            paint_nums++;
16        }
17        paint_type[v[r]]++;
18        while (paint_nums == m) {//双指针这里的while经常会被写成if, 一定要注意
19            if (r - l + 1 < mmin) {
20                a = l;
21                b = r;
22                mmin = r - l + 1;
23            }
24            paint_type[v[l]]--;
25            if (paint_type[v[l]] == 0) {
26                paint_nums--;
27            }
28            l++;
29        }
30    }
31    cout << a << " " << b << endl;
32 }

```

3.6.5 B4006 宝箱

- 题目链接: [B4006 宝箱 \(点击此处\)](#); **注意:** 双指针一定是在有序数组里进行的, 使用双指针前一定要排序!!!

代码示例:

```

1 auto solve() -> void {
2     ll n, k;
3     cin >> n >> k;
4     vector<ll> v(n);
5     for (ll i = 0; i < n; i++) {
6         cin >> v[i];
7     }
8     sort(v.begin(), v.end());//一定要记得
9     ll l = 0, sum = 0;
10    ll sum_max = 0;
11    for (ll r = 0; r < n; r++) {
12        sum += v[r];
13        while (v[r] - v[l] > k) {
14            sum -= v[l];
15            l++;
16        }
17        sum_max = max(sum_max, sum);
18    }
19    cout << sum_max << endl;
20 }

```

3.6.6 P5745 区间最大和

- 题目链接:[P5745 区间最大和 \(点击此处\)](#);

代码示例:

```

1 auto solve() -> void {
2     ll n, m;
3     cin >> n >> m;
4     vector<ll> v(n + 1);
5     for (ll i = 1; i <= n; i++) {
6         cin >> v[i];
7     }
8     ll l = 1, r = 1, mmin = INT_MAX, sum = 0;
9     ll a = 0, b = 0;
10    while (r <= n) { //再次提醒一定不要写成if!!!
11        sum += v[r];
12        while (sum > m) {
13            sum -= v[l];
14            l++;
15        }
16        if (sum == m) {
17            cout << l << " " << r << " " << m << endl;
18            return;
19        }
20        if (m - sum < mmin) {
21            a = l;
22            b = r;
23            mmin = m - sum;
24        }
25        r++;
26    }
27    cout << a << " " << b << " " << m - mmin << endl;
28 }

```

3.6.7 Leetcode3 无重复最长子串

- 题目链接: [无重复最长子串 \(点击此处\)](#);

代码示例:

```

1 class Solution {
2 public:
3     int lengthOfLongestSubstring(string s) {
4         unordered_set<char> window;
5         int left = 0;
6         int maxlen = 0;
7         for (int right = 0; right < s.size(); right++) {
8             char c = s[right];
9             //双指针一般都是使用while判断的,要是发现双指针出错优先检查一下是否把while写成if.
10            //在使用一些容器时灵活使用它的库函数
11            while (window.find(c) != window.end()) {
12                window.erase(s[left]); //这里的erase可以直接对值操作
13                left++;
14            }
15            window.insert(c); //这里的insert还可以用emplace代替
16            maxlen = max(maxlen, right - left + 1);

```

```
17     }
18     return maxlen;
19 }
20 };
```

3.6.8 Leetcode5 最长回文子串

- 题目链接: [最长回文子串 \(点击此处\)](#);

解析: 首先要明确回文串的特点: 要是回文串的长度为奇数, 那么中心字母一定是 1 个 (如: aba). 要是长度为偶数, 那么中心一定是两个相同的字母 (如: abba), 所以我们可以遍历每一个字母并且都把它看作一个回文串的中心, 所以会出现两种情况分别为下面的 1 和 2 代码示例:

```
1 class Solution {
2     public:
3     string longestPalindrome(string s) {
4         if (s.size() < 2) return s;
5         int max_len = 1, start = 0;
6         for (int i = 0; i < s.size(); i++) {
7             //1、回文串长度为奇数, 中心位置在i处。
8             round(s, i, i, start, max_len);
9             //2、回文串长度为偶数, 中心位置在i和i+1的中间。
10            round(s, i, i + 1, start, max_len);
11        }
12        return s.substr(start, max_len);
13    }
14
15    private:
16    void round(const string &s, int l, int r, int &start, int &max_len) {
17        while (l >= 0 && r < s.size() && s[l] == s[r]) {
18            int curr_len = r - l + 1;
19            if (curr_len > max_len) {
20                max_len = curr_len;
21                start = l;
22            }
23            l--;
24            r++;
25        }
26    }
27 };
```

3.7 前缀和以及差分

3.7.1 前缀和模板 (p8218)

- 题目链接: [P821 求区间求区间和 \(点击此处\)](#);

代码示例:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long n;
6     cin >> n;
7     vector<long long> v(n + 1);
8     for (long long i = 1; i <= n; i++) {
9         cin >> v[i];
10    }
11    vector<long long> a(n + 1);
12    a[0] = 0;
13    for (long long i = 1; i <= n; i++) {
14        a[i] = a[i - 1] + v[i];
15    }
16    long long m, sum = 0;
17    cin >> m;
18    while (m--) {
19        long long l, r;
20        cin >> l >> r;
21        sum = a[r] - a[l - 1];
22        cout << sum << endl;
23    }
24    return 0;
25 }

```

3.7.2 差分模板 (P2367)

- 题目链接: [P2367\(语文成绩 \(点击此处\)\)](#);

代码示例:

```

1 void solve() {
2     ll n,p;
3     cin>>n>>p;
4     vector<ll> v(n+1),s(n+1);
5     v[0]=0;
6     for(ll i=1;i<=n;i++){
7         cin>>v[i];
8     }
9     s[0]=0;
10    for(ll i=1;i<=n;i++){
11        s[i]=v[i]-v[i-1];//求差分
12    }
13    while(p--){
14        ll x,y,z;
15        cin>>x>>y>>z;
16        s[x]+=z; //在x到y的区域都加上z
17        if (y + 1 <= n) s[y+1] -= z;//使第y+1的数不变
18    }
19    for(ll i=1;i<=n;i++){
20        v[i]=v[i-1]+s[i];//差分求前缀和的到最终加了z的数组
21    }
22    ll mmin=INT_MAX;

```



```
23     for(ll i=1;i<=n;i++){
24         mmin=min(mmin,v[i]);
25     }
26     cout<<mmin<<endl;
27 }
```

3.7.3 前缀和解决区间重叠问题 (P1047)

- 题目链接:[P1047 校门外的树 \(点击此处\)](#);

代码示例:

```
1 void solve() {
2     ll n, m;
3     cin >> n >> m;
4     while (m--) {
5         ll l, r;
6         cin >> l >> r;
7         v[l] += 1;
8         v[r + 1] -= 1;
9     }
10    for (ll i = 1; i <= n; i++) {
11        v[i] += v[i - 1];
12    }
13    ll ans = 0;
14    for (ll i = 0; i <= n; i++) {
15        if (v[i] == 0) {
16            ans++;
17        }
18    }
19    cout << ans << endl;
20 }
```

3.7.4 P3406 海底高铁

- 题目链接: [P3406 海底高铁 \(点击此处\)](#);

代码示例:

```
1 void solve() {
2     ll n, m;
3     cin >> n >> m;
4     vector<ll> v(m + 1);
5     for(ll i = 1; i <= m; ++i) {
6         cin >> v[i];
7     }
8
9     vector<ll> a(n), b(n), c(n);
10    for(ll i = 1; i <= n - 1; ++i) {
11        cin >> a[i] >> b[i] >> c[i];
12    }
13 }
```

```

14 //求相同路段出现的次数
15 vector<ll> cf(n + 2, 0); //防止越界
16 for(ll i = 1; i < m; i++) {
17     ll l = min(v[i], v[i + 1]);
18     ll r = max(v[i], v[i + 1]);
19     cf[l] += 1;
20     cf[r] -= 1;
21 }
22 for(ll i = 1; i <= n-1; i++) {
23     cf[i] += cf[i - 1]; //对差分求前缀和
24 }
25 //最小花费
26 ll ans = 0;
27 for(ll i = 1; i <= n-1; i++) {
28     if(cf[i] == 0) {
29         continue;
30     }
31     ll cost_no = cf[i] * a[i];
32     ll cost_ic = c[i] + cf[i] * b[i];
33     ans += min(cost_no, cost_ic);
34 }
35 cout << ans << endl;
36 }

```

总结: 其实前缀和和差分一般结合使用, 可用来计算一个区间内重复的次数;

3.7.5 P8830(评委打分)

- 题目链接: [P8830 评委打分 \(点击此处\)](#);

代码示例:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7     int n;
8     cin >> n;
9     vector<int> v(n);
10    for (auto &x: v) {
11        cin >> x;
12    }
13    vector<long long> pre_sum(n + 1);
14    pre_sum[0] = v[0];
15    for (int i = 1; i < n; i++) {
16        pre_sum[i] = pre_sum[i - 1] + v[i]; //提前算好前缀和, 防止TLE
17    }
18    long long max_a, min_a;
19    double aver;
20    max_a = max(v[0], v[1]); //动态维护最大值和最小值
21    min_a = min(v[0], v[1]);
22    for (int i = 2; i < n; i++) {
23        if (v[i] > max_a) max_a = v[i];
24        if (v[i] < min_a) min_a = v[i];
25        aver = (double) (pre_sum[i] - max_a - min_a) / (i - 1);

```

```
26     cout << fixed << setprecision(2) << aver << endl;
27 }
28 return 0;
29 }
```

3.7.6 二维前缀和模板 (P2004)

- 题目链接: [P2004 领地选择 \(点击此处\)](#)

代码示例:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  using i64 = long long;
4  const int N = 1010;
5  i64 v[N][N], a[N][N];
6
7  int main() {
8      i64 n, m, c;
9      cin >> n >> m >> c;
10     for (i64 i = 1; i <= n; i++) {
11         for (i64 j = 1; j <= m; j++) {
12             cin >> v[i][j];
13         }
14     }
15     //计算每个位置的前缀和, 以便计算后面的区间和;
16     for (i64 i = 1; i <= n; i++) {
17         for (i64 j = 1; j <= m; j++) {
18             a[i][j] = a[i - 1][j] + a[i][j - 1] - a[i - 1][j - 1] + v[i][j];
19         }
20     }
21     i64 x1, x2, y1, y2, max = 0, row = 0, col = 0;
22     for (i64 k = 1; k <= m - c + 1; k++) {
23         for (i64 t = 1; t <= n - c + 1; t++) {
24             x1 = k;
25             y1 = t;
26             x2 = x1 + c - 1;
27             y2 = y1 + c - 1;
28             //知道区间的左上角和右下角坐标就可以求这个矩形内的和
29             if ((a[x2][y2] - a[x2][y1 - 1] - a[x1 - 1][y2] + a[x1 - 1][y1 - 1]) > max) {
30                 max = a[x2][y2] - a[x2][y1 - 1] - a[x1 - 1][y2] + a[x1 - 1][y1 - 1];
31                 row = x1;
32                 col = y1;
33             }
34         }
35     }
36     cout << row << " " << col << endl;
37     return 0;
38 }
```

3.7.7 前缀和取模应用 (abc105D)

- 题目链接: [abc105D 糖果分发 \(点击此处\)](#)

重要结论: “两个数取模 M 余数相同, 它们的差一定是 M 的倍数。”; eg: 若有 k 个数取模 M 余数相同, 则 k 个数中任意两个数的差是 M 的倍数, 则一共有 $\binom{k}{2} = \frac{k(k-1)}{2}$ 个区间满足条件

代码示例:

```
1 void solve() {
2     ll n, m, ans = 0;
3     cin >> n >> m;
4     vector<ll> v(n + 1), s(n + 1);
5     for(ll i = 1; i <= n; i++) {
6         cin >> v[i];
7     }
8     s[0] = 0;
9     unordered_map<ll, ll> mp;
10    mp[0] = 1;
11    for(ll i = 1; i <= n; i++) {
12        s[i] = s[i - 1] + v[i];
13        ans += mp[s[i] % m]; //动态计算k(k-1)/2
14        mp[s[i] % m]++;
15    }
16    cout << ans << endl;
17 }
```

3.7.8 二维差分 (P3397)

- 题目链接: [P3397 地毯 \(点击此处\)](#);

代码示例:

```
1 void solve() {
2     ll n, m;
3     cin >> n >> m;
4     vector<vector<ll>> v(n + 2, vector<ll>(n + 2, 0));
5     //求差分与一维类似, 数组开大一圈防越界
6     //已知左上角和右下角坐标
7     for(ll i = 1; i <= m; ++i) {
8         ll x1, y1, x2, y2;
9         cin >> x1 >> y1 >> x2 >> y2;
10        v[x1][y1] += 1;
11        v[x1][y2 + 1] -= 1;
12        v[x2 + 1][y1] -= 1;
13        v[x2 + 1][y2 + 1] += 1;
14    }
15    //在自身求前缀和, 也可以重新开个数组存前缀和
16    for(ll i = 1; i <= n; ++i) {
17        for(ll j = 1; j <= n; ++j) {
18            v[i][j] += v[i - 1][j] + v[i][j - 1] - v[i - 1][j - 1];
19        }
20    }
21 }
```

```
19     }
20 }
21 for(ll i = 1; i <= n; ++i) {
22     for(ll j = 1; j <= n; ++j) {
23         cout << v[i][j] << " ";
24     }
25     cout << "\n";
26 }
27 }
```

3.8 ST 表

ST 表 (Sparse Table) 的适用条件与注意事项

1. 区间必须是静态的 (无修改) ST 表构建完成后不支持任何形式的修改操作 (如单点修改、区间修改)。适合仅进行大量区间查询而不修改数组的场景。
2. 查询函数必须满足可重复贡献 (幂等性 / 结合性) 要求区间函数满足:

$$f(f(a, b), c) = f(a, f(b, c))$$

常见可用的函数:

- 区间最小值 (min)
- 区间最大值 (max)
- 最大公约数 (gcd)
- 最小公倍数 (lcm)
- 按位与、按位或

不适用的函数 (不可分解或会重复计算):

- 区间和 (sum)
- 区间异或 (xor)

3. 数组长度固定, 适合大量查询 预处理时间为

$$O(n \log n),$$

查询时间为

$$O(1).$$

适用于查询数量远大于数组大小的情况。

4. 内存消耗较高 ST 表需要维护一个 $n \times \log n$ 的表格, 空间复杂度为:

$$O(n \log n).$$

5. 不适用的典型场景

- 需要支持修改 (使用线段树更合适)
- 需要区间和、区间加法、区间乘法等复杂操作
- 查询函数不能通过合并两个区间得到

总结 ST 表适用于:

静态数组 + 大量区间查询 + 满足结合律的函数.

3.8.1 ST 表模板题 (P3865)

- 题目链接: [P3865ST 表或 RMQ 问题 \(点击此处\)](#); RMQ 问题 (查询静态区间最大最小值问题);

代码示例:

```

1  void solve() {
2      ll n, m;
3      cin >> n >> m;
4      vector<vector<ll>>> dp(n + 1, vector<ll>(log2(n) + 3));
5      for (ll i = 1; i <= n; i++) {
6          cin >> dp[i][0];
7      }
8      //做ST表
9      for (ll i = 1; i <= 20; ++i) {
10         for (ll j = 1; j + (1 << i) - 1 <= n; ++j) {
11             dp[j][i] = max(dp[j][i - 1], dp[j + (1 << (i - 1))][i - 1]);
12         }
13     }
14     //查询l-r区间内的最大值
15     while (m--) {
16         ll l, r, mmax = 0;

```

```

17         cin >> l >> r;
18         ll len = log2(r - l + 1);
19         mmax = max(dp[l][len], dp[r - (1 << len) + 1][len]);
20         cout << mmax << endl;
21     }
22 }

```

3.8.2 ST 表求最小值和最大值 (综合)

- 题目链接[P2880Balanced Lineup G\(点击此处\)](#);

代码示例:

```

1  void solve() {
2      ll n, q;
3      cin >> n >> q;
4      ll k = log2(n) + 3;
5      vector<vector<ll>> > dp(n + 1, vector<ll>(k));
6      vector<vector<ll>> > mp(n + 1, vector<ll>(k));
7      for (ll i = 1; i <= n; ++i) {
8          cin >> dp[i][0];
9          mp[i][0] = dp[i][0];
10     }
11     //最大值和最小值必须分开做成两张ST表!
12     for (ll i = 1; i <= k; ++i) {
13         for (ll j = 1; j + (1 << i) - 1 <= n; ++j) {
14             dp[j][i] = max(dp[j][i - 1], dp[j + (1 << (i - 1))][i - 1]);
15         }
16     }
17     for (ll i = 1; i <= k; ++i) {
18         for (ll j = 1; j + (1 << i) - 1 <= n; ++j) {
19             mp[j][i] = min(mp[j][i - 1], mp[j + (1 << (i - 1))][i - 1]);
20         }
21     }
22     //查询最大最小;
23     while (q--) {
24         ll l, r, mmin = 0, mmax = 0;
25         cin >> l >> r;
26         ll len = log2(r - l + 1);
27         mmax = max(dp[l][len], dp[r - (1 << len) + 1][len]);
28         mmin = min(mp[l][len], mp[r - (1 << len) + 1][len]);
29         cout << mmax - mmin << endl;
30     }
31 }

```

3.9 动态规划

3.9.1 PTA 探险

- 题目链接: [爱探险 \(点击此处\)](#); 总结: 动态规划的题多积累吧

代码示例:

```

1 void solve() {
2     ll n;
3     cin >> n;
4     vector<ll> v(n + 1);
5     vector<vector<ll>> dp(n + 2, vector<ll>(n + 2));
6     for (ll i = 1; i <= n; ++i) {
7         cin >> v[i];
8     }
9     ll sum = 0;
10    for (ll len = 1; len <= n; len++) {
11        for (ll l = 1; l + len - 1 <= n; ++l) {
12            ll r = l + len - 1;
13            ll best = 0;
14            for (ll i = l; i <= r; ++i) {
15                sum = abs(v[i]);
16                if (v[i] > 0) {
17                    sum += dp[i + 1][r];
18                } else {
19                    sum += dp[l][i - 1];
20                }
21                if (sum > best) {
22                    best = sum;
23                }
24            }
25            dp[l][r] = best;
26        }
27    }
28    cout << dp[1][n] << endl;
29 }

```

3.10 字符串

3.10.1 KMP 算法

- 题目链接: [P3375 【模板】KMP\(点击此处\)](#);

代码示例:

```

1 //KMP的next数组优化, 但是不适合这题输出next数组的步骤
2 // void get_nextval(const string &t, ll nextval[]) {
3     //     ll i = 1, j = 0;
4     //     nextval[1] = 0;
5     //     ll m = (ll) t.size() - 1;
6     //     while (i <= m) {
7         //         if (j == 0 || t[i] == t[j]) {
8             //             ++i;
9             //             ++j;
10            //             if (t[i] != t[j]) {
11                //                 nextval[i] = j;
12            //             } else {
13                //                 nextval[i] = nextval[j];
14            //             }
15            //         } else {

```



```

16         //         j = nextval[j];
17         //     }
18     // }
19 // }
20 void get_next(const string &t, ll next[]) {
21     ll i = 1, j = 0;
22     next[1] = 0;
23     ll m = (ll) t.size() - 1;
24     while (i <= m) {
25         if (j == 0 || t[i] == t[j]) {
26             ++i;
27             ++j;
28             next[i] = j;
29         } else {
30             j = next[j];
31         }
32     }
33 }
34
35 void set_KMP(const string &s, const string &t, ll next[]) {
36     ll i = 1, j = 1;
37     ll n = (ll) s.size() - 1;
38     ll m = (ll) t.size() - 1;
39     while (i <= n && j <= m) {
40         if (j == 0 || s[i] == t[j]) {
41             ++i;
42             ++j;
43         } else {
44             j = next[j];
45         }
46         if (j > m) {
47             cout << i - m << endl;
48             j = next[j];
49         }
50     }
51 }
52
53 auto solve() -> void {
54     string s, t;
55     cin >> s >> t;
56     s = " " + s;
57     t = " " + t;
58     ll *next = new ll[t.size() + 1];
59     get_next(t, next);
60     set_KMP(s, t, next);
61     for (ll i = 2; i <= t.size(); i++) {
62         cout << next[i] - 1 << " ";
63     }
64     delete[] next;
65 }

```

3.11 有关二进制的习题总结

3.11.1 2026 天梯选拔 1-8

- 题目链接: [1-8 究极同化 \(点击此处\)](#);

总结: 本题提供了两种选择, 一个是按位或 ($|$: 只要有一个 1 结果就是 1), 一个是按位

与 (&: 两个都为 1 结果才为 1); 要使和最大及让高位二进制尽可能的多 1;

代码示例:

```

1 auto solve() -> void {
2     ll n;
3     cin >> n;
4     vector<ll> v(25, 0);
5     for (ll i = 0; i < n; i++) {
6         ll x;
7         cin >> x;
8         for (ll j = 0; j < 25; j++) {
9             if ((x >> j) & 1) {
10                 v[j]++; //记录每个二进制位1的个数;
11             }
12         }
13     }
14     ll ans = 0;
15     for (ll i = 0; i < n; i++) {
16         ll cnt = 0;
17         for (ll j = 0; j < 25; j++) {
18             if (v[j] > 0) {
19                 cnt |= (1LL << j); //把第j位赋上1;
20                 v[j]--;
21             }
22         }
23         ans = (ans + (cnt * cnt)) % MOD;
24     }
25     cout << ans << endl;
26 }

```

3.11.2 2025 天梯赛复赛 7-5

- 题目链接: [7-5 异或和 \(点击此处\)](#);

代码示例:

```

1 auto solve() -> void {
2     ll n;
3     cin >> n;
4     vector<ll> v(n);
5     for (ll i = 0; i < n; i++) {
6         cin >> v[i];
7     }
8     ll x = 0, sum_min = 0;
9     for (ll i = 0; i < 32; i++) {
10         ll count1 = 0;
11         for (ll j = 0; j < n; j++) {
12             if ((v[j] >> i) & 1) {
13                 count1++;
14             }
15         }
16         ll count0 = n - count1; //想象成1-32为首行然后n个数以二进制的形式依次排在后续行中,
17                                 //每个数的1-32位二进制与首行一一对应
18                                 //所以0的个数就是总行数n-count1
19         if (count1 >= count0) {

```

```

20 //因为x要尽量大, 所以当cnt1>cnt0时, 让x的i位赋上1; 这样既可以保证
21 //x本身增大又由于异或的性质(相同为0, 不同为1)使这个数异或上x后变小
22 //之后之前的0会变成1所有sum_min=0的个数乘上它的二进制权位
23     x |= (1LL << i);
24     sum_min += count0 * (1LL << i);
25 } else {
26 //当0比较多时, x本来就是0, 所有不需要按位或运算,
27 //而此时原本的1还是1, 所以sum_min=1的个数乘上它的二进制权位
28     sum_min += count1 * (1LL << i);
29 }
30 }
31 cout << x << endl;
32 cout << sum_min << endl;
33 }

```

情况	决策	异或后该位的 1 数量	对 <code>sum_min</code> 的贡献
$count1 \geq count0$	令 $x_i = 1$	$count0$ (原本的0变1)	$count0 \times 2^i$
$count1 < count0$	令 $x_i = 0$	$count1$ (原本的1没变)	$count1 \times 2^i$

3.11.3 P1062 数列

- 题目链接:[P1062 数列 \(点击此处\)](#); 新颖之处: 二进制求解; 因为题目是选若干个 k 的幂的组合, 所以可以用二进制的 0 和 1 做开关 (每个幂都有选或不选两种选择就像二进制的 0 和 1 一样) 所以**第 N 项 = 用 N 的第二进制当做选幂的开关**

代码示例:

```

1 void solve() {
2     ull k, n;
3     cin >> k >> n;
4     ull ans = 0, power = 1;
5     while (n > 0) {
6         if (n & 1) { //看n的二进制最后一位是否是1, 是就加幂;
7             ans += power;
8         }
9         power *= k;
10        n >>= 1; //右移更新最后一位二进制, 丢掉原来的最后一位
11    }
12    cout << ans << endl;
13 }

```

3.12 快速幂

3.12.1 2026 天梯赛选拔 1-7

- 题目链接: [1-7e 的 7777777\(点击此处\)](#);

数学原理: 一个由 n 个 7 组成的数字可以写成如下等比数列求和的形式:

$$777\dots 7(n\text{个}) = 7 \times (1 + 10^1 + 10^2 + \dots + 10^{n-1})$$

利用等比数列求和公式 $S_n = \frac{a_1(q^n - 1)}{q - 1}$, 其中首项 $a_1 = 1$, 公比 $q = 10$:

$$111\dots 1(n\text{个}) = \frac{10^n - 1}{10 - 1} = \frac{10^n - 1}{9}$$

所以, 最终需要计算的表达式为:

$$\text{aares} = \left(7 \times \frac{10^n - 1}{9} \right) \pmod{10^9 + 7}$$

由此数学原理的出来的最终公式可看出要使用快速幂求次方以及要利用费马小定理解题;

代码示例:

```

1 ll qpow(ll a, ll b) {
2     ll ans = 1;
3     a %= MOD;
4     while (b > 0) {
5         if (b & 1) ans = (ans * a) % MOD;
6         a = (a * a) % MOD;
7         b >>= 1;
8     }
9     return ans;
10 }
11
12 ll feim(ll a) {
13     return qpow(a, MOD - 2);
14 }
15
16 auto solve() -> void {
17     ll n;
18     cin >> n;
19     ll fz = qpow(10, n);
20     fz = (fz - 1 + MOD) % MOD;
21     ll fm = feim(9);
22     ll ans = (7 * fz) % MOD;
23     ll cnt = (ans * fm) % MOD;

```

```
24     cout << cnt << endl;
25 }
```

3.13 单调栈

3.13.1 P5788 单调栈模板

- 题目链接: [P5788 单调栈 \(点击此处\)](#);

代码示例:

```
1 auto solve() -> void {
2     ll n;
3     if (!(cin >> n)) return;
4     vector<ll> v(n + 1), res(n + 1, 0);
5     for (ll i = 1; i <= n; i++) {
6         cin >> v[i];
7     }
8     stack<ll> st;
9     for (ll i = 1; i <= n; i++) {
10         while (!st.empty() && v[i] > v[st.top()]) {
11             res[st.top()] = i;
12             st.pop();
13         }
14         st.emplace(i);
15     }
16     for (ll i = 1; i <= n; i++) {
17         cout << res[i] << " ";
18     }
19 }
```

3.13.2 P1318 积水面积

- 题目链接: [P1318 积水面积 \(点击此处\)](#); **注:** 这题在双指针那里也有讲解, 双指针是纵向相加, 而单调栈是横向相加。

代码示例:

```
1 class Solution {
2     public:
3     int trap(vector<int>& h) {
4         int n = h.size();
5         stack<int> s;
6         int ans = 0;
7         for (int i = 0; i < n; i++) {
8             while (!s.empty() && h[i] > h[s.top()]) {
9                 int di = s.top();
10                s.pop();
11                if (s.empty()) break;
```

```

12         int left = s.top();
13         //s.pop()这一步非常致命, 千万不要把左边删掉了, 只是借用来求最小高度, 不是踢出栈外
14         int w=i-left-1;
15         ans += (min(h[left], h[i]) - h[di])*w;
16     }
17     s.emplace(i);
18 }
19 return ans;
20 }
21 };

```

3.14 其他 (Luogu 题库)

3.14.1 P3741 凑 VK

- 题目链接: [P3741 凑 VK\(点击此处\)](#); 方法见 Codeforces266B;

代码示例:

```

1 void solve() {
2     ll n, count = 0;
3     string s;
4     cin >> n >> s;
5     for (ll i = 0; i < n - 1; i++) {
6         if (s[i] == 'V' && s[i + 1] == 'K') { //只有有两个相同的就可以凑出一个VK
7             count++;
8             s[i]='X';
9             s[i+1]='X';
10        }
11    }
12    for (ll i=0;i<n-1;i++) {
13        if (s[i]==s[i+1]&&s[i]!='X') {
14            cout<<count+1<<endl;
15            return;
16        }
17    }
18    cout<<count<<endl;
19 }

```

3.14.2 B3627 立方根 (cbirt() 函数)

- 题目链接: [B3627 立方根 \(点击此处\)](#); 用库函数 cbirt();

代码示例:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 int main() {
4     long long n,x;
5     cin>>n;

```

```
6     x=(long long)(cbrt(n)+1e-9);//防止形如9.999999的数字被误判为9
7     //不是为了四舍五入, 而是为了确保接近整数但略小时能被正确取整;
8     cout<<x<<endl;
9     return 0;
10 }
```

3.14.3 P1765 手机键盘

- 题目链接: [P1765 手机键盘 \(点击此处\)](#); 总结: 一定要学会打表, 像这题还有类似剪刀石头布的那种题, 打表最方便;

代码示例:

```
1 int main() {
2     ios_base::sync_with_stdio(false);
3     cin.tie(nullptr);
4     int a[26] = {1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 4, 1, 2, 3, 1,
5                 2, 3, 4};
6     string s;
7     int count = 0;
8     getline(cin, s);
9     for (int i = 0; i < s.size(); i++) {
10         if (s[i] >= 'a' && s[i] <= 'z') {
11             count += a[s[i] - 'a'];
12         } else {
13             count++;
14         }
15     }
16     cout << count << endl;
17     return 0;
18 }
```

3.14.4 P5734 字符串的连接, 插入, 截取和查找

- 题目链接: [P5734 文件处理软件 \(点击此处\)](#)模板题熟悉字符串;

代码示例:

```
1 int main() {
2     int t, n;
3     string s;
4     cin >> t >> s;
5     while (t--) {
6         cin >> n;
7         if (n == 1) {
8             string s1;
9             cin >> s1;
10            s += s1;
11            cout << s << endl;
12        }
13        if (n == 2) {
```

```

14     int a, b;
15     cin >> a >> b;
16     s = s.substr(a, b);
17     cout << s << endl;
18 }
19 if (n == 3) {
20     int a;
21     string s2;
22     cin >> a >> s2;
23     s.insert(a, s2);
24     cout << s << endl;
25 }
26 if (n == 4) {
27     string s3;
28     cin >> s3;
29     int pos = s.find(s3);
30     cout << (pos != string::npos ? pos : -1) << endl;
31 }
32 }
33 return 0;
34 }

```

3.14.5 P1957 区分输入个数以及保存上次输入

- 题目链接: [P1957 口算练习题 \(点击此处\)](#); 总结: 可以用字符串流和数组来区分输入个数, 用 bool 类型来保存上一次输入;

代码示例:

```

1 int main() {
2     int i;
3     cin >> i;
4     char opr = '+';
5     cin.ignore();
6     while (i--) {
7         string s;
8         getline(cin, s);
9         stringstream ss(s); //按空格分割字符串并存入数组
10        vector<string> v;
11        while (ss >> s) {
12            v.push_back(s); //若输入三个则v[0]是字符, 若输入2个则v[0]是数字
13        }
14        bool flag = false;
15        if (v[0] == "a") {
16            opr = '+';
17            flag = true; //true则更新符号, false则保留上一次符号
18        } else if (v[0] == "b") {
19            opr = '-';
20            flag = true;
21        } else if (v[0] == "c") {
22            opr = '*';
23            flag = true;
24        }
25        string ans;
26        if (flag) { //说明v[0]是符号
27            ans = v[1] + opr + v[2] + "=";
28            if (opr == '+') {

```



```

29         ans += to_string(atoi(v[1].c_str()) + atoi(v[2].c_str()));
30     } else if (opr == '-') {
31         ans += to_string(atoi(v[1].c_str()) - atoi(v[2].c_str()));
32     } else if (opr == '*') {
33         ans += to_string(atoi(v[1].c_str()) * atoi(v[2].c_str()));
34     }
35 } else {
36     ans = v[0] + opr + v[1] + "=";
37     if (opr == '+') {
38         ans += to_string(atoi(v[0].c_str()) + atoi(v[1].c_str()));
39     } else if (opr == '-') {
40         ans += to_string(atoi(v[0].c_str()) - atoi(v[1].c_str()));
41     } else if (opr == '*') {
42         ans += to_string(atoi(v[0].c_str()) * atoi(v[1].c_str()));
43     }
44 }
45 cout << ans << endl;
46 cout << ans.size() << endl;
47 }
48 }

```

3.14.6 P1614 队列实现循环

- 题目链接: [P1614 爱与愁的心痛 \(点击此处\)](#);

代码示例:

```

1  int main() {
2      i32 n,m,t,sum=0;
3      cin>>n>>m;
4      queue<i32> q;
5      i32 min=INT_MAX;
6      for (int i=0;i<n;i++) {
7          cin>>t;
8          q.push(t);
9          if (q.size()==m) {
10             sum=0;
11             for (int j=0;j<m;j++) {
12                 sum+=q.front();
13                 q.push(q.front()); //将每次用过的头放到对尾, 保证了最后队列的顺序不变;
14                 q.pop();
15             }
16             if (sum<min) {
17                 min=sum;
18             }
19             q.pop(); //删除第一组的第一个元素实现向后推进
20         }
21     }
22     cout<<min<<endl;
23     return 0;
24 }

```

此题也可用前缀和实现:

代码示例:

```

1 int main() {
2     i32 n, m;
3     cin >> n >> m;
4     vector<i32> v(n + 1);
5     for (i32 i = 1; i <= n; i++) {
6         cin >> v[i];
7     }
8     vector<i32> s(n + 1);
9     s[0] = 0;
10    for (i32 i = 1; i <= n; i++) {
11        s[i] = s[i - 1] + v[i];
12    }
13    i32 min = INT_MAX;
14    for (int i = 1; i <= n - m + 1; i++) {
15        if (s[i + m - 1] - s[i - 1] < min) {
16            min = s[i + m - 1] - s[i - 1];
17        }
18    }
19    cout << min << endl;
20    return 0;
21 }

```

3.14.7 P1161 开关和向下取整问题

- 题目链接: [P1161 开灯 \(点击此处\)](#)

代码示例:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using i32 = long long;
4 #define N 2000005
5 i32 a[N];
6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    i32 n;
11    cin >> n;
12    while (n--) {
13        double x;
14        i32 t;
15        cin >> x >> t;
16        for (i32 i = 1; i <= t; i++) {
17            if (a[(int) (x * i)] == 0) {
18                a[(int) (x * i)] = 1;
19            } else {
20                a[(int) (x * i)] = 0;
21            }
22        }
23    }
24    for (i32 i = 0; i < N; i++) {
25        if (a[i]) {
26            cout << i << endl;
27        }
28    }

```

```

29     return 0;
30 }

```

3.14.8 负数的进制转换

- 题目链接: [P1017 进制转换 \(点击此处\)](#);

代码示例:

```

1 string tobase(ll n, ll R) {
2     if (n == 0) return "0";
3     vector<ll> digits;
4     while (n) {
5         ll r = n % R;
6         if (r < 0) { //当n或者R是负数的时候, 余数r可能会被计算为负数
7             r += abs(R); //其实就是被除数=除数*商+余数的过程
8             //n+=1      //这样写也可以, 在纸上算一下就知道了
9             n = (n - r) / R; //因为R变成正数所以商也要做出改变
10        } else {
11            n /= R;
12        }
13        digits.push_back(r);
14    }
15    string rem;
16    for (ll i = digits.size() - 1; i >= 0; i--) {
17        ll d = digits[i];
18        if (d < 10) rem += to_string(d);
19        else rem += 'A' + d - 10;
20    }
21    return rem;
22 }
23 void solve() {
24     ll n, R;
25     cin >> n >> R;
26     string s;
27     s = tobase(n, R);
28     cout << to_string(n) << "=" << s << "(base" << R << ")" << endl;
29 }

```

3.14.9 P1061 循环递增字母序列

- 题目链接: [p1061Jam 的计数法 \(点击此处\)](#);

代码示例:

```

1 void solve(){
2     ll s,t,w;
3     string st;
4     cin>>s>>t>>w>>st;
5     for(ll i=1;i<=5;++i){
6         for(ll j=w-1;j>=1;--j){
7             if(st[j]-'a'+1<=t-(w-j)){//防止最后几位加1时越界

```

```

8         st[j]++;
9         for(11 k=j+1;k<w;++k){
10             st[k]=st[k-1]+1;
11         }
12         cout<<st<<endl;
13         break;
14     }
15 }
16 }
17 }

```

3.14.10 字串成比例数字查找

- 题目链接: [P1008 三连击 \(点击此处\)](#); 总结: 这是技巧性的东西, 可以积累一下。

代码示例:

```

1 bool ok(11 a, 11 b, 11 c) {
2     string s = to_string(a) + to_string(b) + to_string(c);
3     if (s.size() != 9)return false;
4     sort(s.begin(), s.end());
5     return s == "123456789";
6 }
7
8 void solve() {
9     11 a, b, c;
10    for (a = 160; a <= 360; ++a) {//这个范围可通过暴力打表找出
11        b = a * 2;
12        c = a * 3;
13        if (ok(a, b, c)) {
14            cout << a << " " << b << " " << c << endl;
15        }
16    }
17 }

```

暴力打表: 对这题而言即用全排列列出所有可能然后输出满足条件的数

3.14.11 解二元一次方程组 (sscanf 的用法)

- 解释: 此题为偶然间发现的题并非来自做题网站但是题目很直接。

C++ 中的 sscanf 深度解析

1. 核心定义与引入

在 C++ 中, 推荐包含 `<cstdio>` 而非 `<stdio.h>`。虽然 `sscanf` 是 C 风格的函数, 但在处理格式化字符串解析时, 其性能通常优于 `stringstream`。

```
int sscanf(const char *buffer, const char *format, ...);
```

注意： `sscanf` 只能直接操作 `const char*`。如果你使用的是 `std::string`，必须调用 `.c_str()` 方法。

2. 格式化匹配高级技巧

`sscanf` 的强大之处在于其格式控制符的组合能力：

- 忽略特定项 (`%*`): 使用 `%*d` 或 `%*s` 可以跳过不需要的数据，不进行赋值。
- 字符集匹配 (`%[ ]`):
 - * `%[a-z]`: 匹配任意小写字母。
 - * `%[^\n]`: 读取直到换行符为止（常用于读取带空格的整行）。
- 宽度限制: `%10s` 限制最多读取 10 个字符，防止缓冲区溢出。

3. 典型应用场景示例

任务	格式字符串	说明
解析 IP 地址	<code>"%d.%d.%d.%d"</code>	将点分十进制解析为 4 个整数
提取括号内容	<code>"%*[^()](%[^()])"</code>	跳过左括号前的内容，提取括号内内容
过滤非数字	<code>"%*[^0-9]%d"</code>	跳过前导非数字字符，读取第一个数字

4. C++ 实践代码片段

```
#include <cstdio>

#include <string>

#include <vector>

void demo() {

    std::string data = "ID:1024, Score:98.5, Name:Alice";

    int id;

    float score;
```

```

char name[20];

// C++ 中必须使用 .c_str()
int count = sscanf(data.c_str(), "ID:%d, Score:%f, Name:%s", &id, &score, name);

if (count == 3) {
    // 成功解析所有字段
}
}

```

5. sscanf vs std::stringstream

- 性能: `sscanf` 通常比 `std::stringstream` 快 3-10 倍。
- 安全性: `stringstream` 更安全, 会自动处理内存; `sscanf` 需要预先分配 `char` 数组, 存在溢出风险。
- 灵活性: `sscanf` 的模式匹配 (如 `%[~,]`) 在处理复杂分隔符时比 `getline` 更简洁。

6. 常见陷阱 (Pitfalls)

1. 指针传递: 必须传递变量地址 (`&var`), 否则会导致段错误 (Segmentation Fault)。
2. 数组越界: 解析字符串到 `char[]` 时, 务必指定最大宽度, 如 `%19s` 对应 `char[20]`。
3. 返回值检查: 始终检查返回值是否等于预期的参数个数, 以确保数据的完整性。

代码示例:

```

1 class System {
2     private:
3         double a = 0, b = 0, c = 0, d = 0, e = 0, f = 0;
4
5     public:
6         auto input() -> void {
7             string s;
8             getline(cin, s);
9             ll count = sscanf(s.c_str(), "%lfx+%lfy=%lf;%lfx+%lfy=%lf", &a, &b, &c, &d,
10                             &e, &f);
11             if (count < 6) {
12                 cout << "警告: 输入格式解析不完整, 请确保包含所有系数和符号!" << endl;
13             }
14         }
15     }

```

```

14
15     auto solve() -> void {
16         double D = a * e - b * d;
17         if (D != 0) {
18             const double x = (c * e - b * f) / D;
19             const double y = (a * f - c * d) / D;
20             printf("方程组的解为: x = %.2f, y = %.2f\n", x, y);
21         } else {
22             if ((a * f - c * d) == 0 && (c * e - b * f) == 0) {
23                 cout << "方程组有无数个解。" << endl;
24             } else {
25                 cout << "方程组无解。" << endl;
26             }
27         }
28     }
29 };
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34     System sys;
35     sys.input();
36     sys.solve();
37     return 0;
38 }

```

3.14.12 P5250 set 的用法

- 题目链接:[P5250 木材仓库 \(点击此处\)](#); STL 的用法必须积累和记忆, 非常重要!

代码示例:

```

1 auto solve() -> void {
2     ll n;
3     cin >> n;
4     set<ll> st;
5     while (n--) {
6         ll q, l;
7         cin >> q >> l;
8         if (q == 1) {
9             if (st.count(l)) cout << "Already Exist" << endl;
10            else st.insert(l);
11        } else {
12            if (st.empty()) {
13                cout << "Empty" << endl;
14            } else {
15                //lower_bound函数是用在数组vector里面的，但是在set中是它自配的函数表示
16                //第一个>=l的数的下标。
17                //在vector中为id=lower_bound(v.begin(),v.end(),x)-v.begin();
18                auto it = st.lower_bound(l);
19                //若it==st.begin()说明所有数都大于l
20                if (it == st.begin()) {
21                    cout << *it << endl;
22                    st.erase(it);
23                } //若it==st.end()说明所有数都小于l
24                else if (it == st.end()) {
25                    it--;
26                    cout << *it << endl;

```

```

27         st.erase(it);
28     } else {
29         //prev(it)是指it迭代器所指元素的上一个元素, 即最后一个<l的数。
30         auto pre = prev(it);
31         ll dist_pre = l - *pre;
32         ll dist_next = *it - l;
33         if (dist_pre > dist_next) {
34             cout << *it << endl;
35             st.erase(it);
36         } else {
37             cout << *pre << endl;
38             st.erase(pre);
39         }
40     }
41 }
42 }
43 }
44 }

```

3.14.13 P1056 排座位

- 题目链接: [P1056 排座位 \(点击此处\)](#);

代码示例:

```

1 //用结构体是为了方便后续的排序, 也可换成vector<pair<ll,ll>>.
2 struct med {
3     ll num, p;
4 } row[1005], col[1005];
5 //使结构体在前k个横列和前l个纵列可以分开的对数最多
6 auto cmp1(const med &a, const med &b) -> bool {
7     return a.num > b.num;
8 }
9 //为了满足序号从小到大
10 auto cmp2(const med &a, const med &b) -> bool {
11     return a.p < b.p;
12 }
13
14 auto solve() -> void {
15     ll m, n, k, l, d;
16     cin >> m >> n >> k >> l >> d;
17     for (ll i = 0; i < d; i++) {
18         ll x1, y1, x2, y2;
19         cin >> x1 >> y1 >> x2 >> y2;
20         if (x1 == x2) {
21             //col[]内部的下标是为了防止后续同样的纵列依旧新增数组大小
22             //目的是吧同一个纵列或行列所能分开的对数最大化(贪心思想)
23             col[min(y1, y2)].p = min(y1, y2);
24             col[min(y1, y2)].num++;
25         } else {
26             row[min(x1, x2)].p = min(x1, x2);
27             row[min(x1, x2)].num++;
28         }
29     }
30     sort(row, row + m, cmp1);
31     sort(col, col + n, cmp1);
32     sort(row, row + k, cmp2);
33     sort(col, col + l, cmp2);

```



```

34     for (ll i = 0; i < k; i++) {
35         cout << row[i].p << " ";
36     }
37     cout << endl;
38     for (ll i = 0; i < l; i++) {
39         cout << col[i].p << " ";
40     }
41 }

```

3.14.14 P1072 最大公因数 (约数) 和最小公倍数

- 题目链接: [P1072 趣味数学 \(点击此处\)](#); x 和 b_0 的最小公倍数是 b_1 说明 x 和 b_0 都是 b_1 的因数 (约数);

约数 (因数): 定义: 如果整数 a 除以整数 b 的余数为 0, 那么 b 就是 a 的约数。例子:

– 6 的约数有 1, 2, 3, 6。

最大公因数: 定义: 两个或多个整数共有约数中最大的那个, 称为最大公约数 (gcd) 例子:

– 12 的约数集合为 $A = \{1, 2, 3, 4, 6, 12\}$

– 18 的约数集合为 $B = \{1, 2, 3, 6, 9, 18\}$

最小公倍数: 定义: 两个或多个整数公有的倍数中最小的正整数, 称为最小公倍数 (lcm);

– 4 的倍数有 $\{4, 8, 12, 16, \dots\}$, 6 的倍数有 $\{6, 12, 18, \dots\}$ 。他们最小公共倍数是 12。

– 数学公式: $\text{lcm}(a, b) = \frac{|a \times b|}{\text{gcd}(a, b)}$ 。

代码示例:

```

1  auto lcm(const ll a, const ll b) -> ll {
2      return a / gcd(a, b) * b;
3  }
4
5  auto solve() -> void {
6      ll cnt = 0;
7      ll a0, a1, b0, b1;
8      cin >> a0 >> a1 >> b0 >> b1;
9      for (ll x = 1; x * x <= b1; x++) {
10         if (b1 % x == 0) {
11             if (gcd(a0, x) == a1 && lcm(x, b0) == b1) {
12                 cnt++;
13             }
14             //注意这里之前说过: 若x是b1的因数, 则b1/x也是b1的因数
15             ll d = b1 / x;
16             if (d != x) {
17                 if (gcd(a0, d) == a1 && lcm(d, b0) == b1) {

```

```

18         cnt++;
19     }
20 }
21 }
22 }
23 cout << cnt << endl;
24 }

```

3.14.15 康托尔折线 (z 字形—蛇形矩阵遍历)

- 题目链接:[P1014 Cantor 表 \(点击此处\)](#); **注意**: 这里的蛇形遍历是指对角线的蛇形而不是行或者列的蛇形。代码示例:

```

1 auto solve() -> void {
2     ll n;
3     cin >> n;
4     ll row = 1;
5     while (n > row) {
6         n -= row;
7         row++;
8     }
9     if (row & 1) {
10        cout << (row - n + 1) << "/" << n << endl;
11    } else {
12        cout << n << "/" << (row - n + 1) << endl;
13    }
14 }

```

3.14.16 异或和的特性及用法

- 题目链接: [P1469 找筷子 \(点击此处\)](#); **注意**: 此题内存给的极少, 但是数据范围很大, 不能开 map 等其他容器;

原理 (异或的性质):

异或运算 (XOR) 的核心特性

异或运算 (在 C++ 中为 `^`) 具有以下三个对本题至关重要的特性:

1. 自反性: 任何数与自身异或结果为 0。

$$x \oplus x = 0$$

2. 恒等律：任何数与 0 异或结果保持不变。

$$x \oplus 0 = x$$

3. 结合律与交换律：运算顺序不影响结果，成对的数可以抵消。

$$a \oplus b \oplus a = (a \oplus a) \oplus b$$

$$= 0 \oplus b$$

$$= b$$

代码示例：

```
1 auto solve() -> void {
2     ll n;
3     cin >> n;
4     ll ans = 0;
5     for (ll i = 0; i < n; i++) {
6         ll x;
7         cin >> x;
8         ans ^= x;
9     }
10    cout << ans << endl;
11 }
```

4 lanqiao 刷题总结

在本节中，我将总结我在 lanqiao 上做题的经验。

4.1 遇到的问题以及学到的方法

4.1.1 最少回文子串

- 题目链接:[最少子串回文数 \(点击此处\)](#); 方法: 1、**回文数特点 (非常重要!!!)**:最多允许一个字符出现奇数次 (要么所有字符的出现次数都为偶数, 要么只能有一个字符出现次数为奇数 (**出现在中心位置**)); 2、总结: 若所有字符出现次数均为偶数则答案为 1; 否则答案是出现次数为奇数的字符的个数;

代码示例:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main(){
4     int n;
5     string s;
6     cin>>n>>s;
7     vector<int> v(26,0);
8     for(auto x:s){
9         v[x-'a']++;
10    }
11    int odd_cnt=0;
12    for(auto ch:v){
13        if(ch&1){
14            odd_cnt++;
15        }
16    }
17    if(odd_cnt==0) cout<<1<<endl;
18    else cout<<odd_cnt<<endl;
19    return 0;
20 }
```

4.1.2 合理配置

- 题目链接 [合理配置 \(点击此处\)](#); 思路用总的袋子数-合理配置的数量 = 幽默配置的数量;

代码示例:

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4 using ll=long long;
5 int main(){
6     ll a,b;
7     cin>>a>>b;
8     ll ans=INT_MAX;
9     for(ll i=0;i<=min(a,b);i++){
10         ans=min(ans,abs(a+b-4*i));
11     }
12     cout<<ans<<endl;
13     return 0;
14 }
```

5 PTA 刷题总结

在本节中，我将总结我在 PTA 上做题的经验。

5.1 遇到的问题以及学到的方法

5.2 对数组每一列按升序排序

- 题目链接: [坚固矩阵 \(点击此处\)](#);

代码示例:

```
1 void solve() {
2     ll n, m;
3     cin >> n >> m;
4     vector<vector<ll>> > v(n, vector<ll>(m));
5     for (ll i = 0; i < n; i++) {
6         for (ll j = 0; j < m; j++) {
7             cin >> v[i][j];
8         }
9     }
10    for (ll j = 0; j < m; j++) {
11        //开一个新数组来存每一列的数字，然后排序;
12        vector<ll> a;
13        for (ll i = 0; i < n; i++) {
14            a.push_back(v[i][j]);
15        }
16        sort(a.begin(), a.end());
17        //将排序好的数组在放进原数组中;
18        for (ll i = 0; i < n; i++) {
19            v[i][j] = a[i];
20        }
21    }
22    bool flag;
23    for (ll i = 0; i < n; i++) {
24        flag = true;
25        for (ll j = 0; j < m; j++) {
```

```

26         cout << (flag == true ? "" : " ") << v[i][j];
27         flag = false;
28     }
29     cout << endl;
30 }
31 }

```

5.3 轮流问题

- 题目链接: [小杭玩游戏 \(点击此处\)](#); 总结: 可以用 bool 来表示轮到谁了。

代码示例:

```

1 void solve() {
2     ll a,b,n,cnt=0;
3     cin>>a>>b>>n;
4     bool flag=true;
5     while(n>0){
6         ll m;
7         cnt++;
8         if(flag){
9             m=gcd(n,a);
10        }else{
11            m=gcd(n,b);
12        }
13        n-=m;
14        flag=!flag;
15    }
16    if(flag){
17        cout<<"Xiao Hong Win"<<endl;
18        cout<<cnt<<endl;
19    }else{
20        cout<<"Xiao Ming Win"<<endl;
21        cout<<cnt<<endl;
22    }
23 }

```

5.4 结构体的使用

- 题目链接: [L2-003 月饼 \(点击此处\)](#);

总结: 当要记录多个数据 (两个以上) 时可以用结构体来操作。处理两个数据可以用哈希表或者其他容器操作。

lambda 函数也要了解一下。

代码示例:

```

1 class total {
2     public:

```

```

3     double w;
4     double v;
5     double d;
6 };
7
8 bool cmp(total t1, total t2) {
9     return t1.d > t2.d;
10 }
11
12 void solve() {
13     ll n;
14     double m;
15     cin >> n >> m;
16     vector<total> a(n);
17     for (ll i = 0; i < n; i++) {
18         cin >> a[i].w;
19     }
20     for (ll i = 0; i < n; i++) {
21         cin >> a[i].v;
22         a[i].d = a[i].v / a[i].w;
23     }
24     sort(a.begin(), a.end(), cmp);
25     //也可以写成lambda函数的形式
26     // sort(a.begin(), a.end(), [](total &t1, total &t2){
27         //     return t1.d > t2.d;
28     // });
29     double ans = 0;
30     for (ll i = 0; i < n && m > 0; i++) {
31         if (a[i].w <= m) {
32             ans += a[i].v;
33             m -= a[i].w;
34         } else {
35             ans += a[i].d * m;
36             m = 0;
37         }
38     }
39     cout << fixed << setprecision(2) << ans << endl;
40 }

```

5.5 固定位数问题

- 题目链接: [出生年 \(点击此处\)](#) 总结: set 去重可以检查有几个不同, for 循环固定 4 次, 就算 temp=0 也要继续循环, 保证了始终有四位。

代码示例:

```

1 int main() {
2     int y, n;
3     cin >> y >> n;
4     for (int i = y; ; i++) {
5         set<int> st;
6         int temp = i;
7         for (int j = 0; j < 4; j++) {
8             st.insert(temp % 10);
9             temp /= 10;
10        }

```

```

11     if (st.size() == n) {
12         printf("%d %04d", i - y, i);
13         return 0;
14     }
15 }
16 }

```

5.5.1 字符串查找连续满足条件串的方法

- 题目链接: [L1-058 翻了 \(点击此处\)](#);

代码示例:

```

1 auto solve() -> void {
2     string s;
3     getline(cin, s);
4     for (ll i = 0; i < s.size(); i++) {
5         if (s[i] == '6') {
6             ll cnt = 0;
7             ll j = i;
8             //改成while循环代替之前的cnt=0和!=0的分类讨论
9             //尽量用这种思想替换掉以取的思想
10            while (j < s.size() && s[j] == '6') {
11                cnt++;
12                j++;
13            }
14            if (cnt > 9) {
15                cout << "27";
16            } else if (cnt > 3) {
17                cout << "9";
18            } else {
19                for (ll k = 0; k < cnt; k++) {
20                    cout << "6";
21                }
22            }
23            i = j;
24        } else {
25            cout << s[i];
26            i++;
27        }
28    }
29 }

```

5.5.2 活用字符串子串提取函数 (substr)

- 题目链接: [L1-059 敲笨钟 \(点击此处\)](#);

代码示例:

```

1 auto solve() -> void {
2     string s;
3     getline(cin, s);

```



```

4 //要熟练的使用find和substr函数，对字符串的提取和替换都有很大的作用
5 size_t pos1 = s.find(',');
6 size_t pos2 = s.find('.');
7 bool flag = false;
8 if (pos1 >= 3 && pos2 >= 3) {
9     if (s.substr(pos1 - 3, 3) == "ong" && s.substr(pos2 - 3, 3) == "ong") {
10         flag = true;
11     }
12 }
13 if (!flag) {
14     cout << "Skipped" << endl;
15 } else {
16     ll i = s.size() - 1;
17     ll cnt = 0;
18     while (i > 0) {
19         if (s[i] == ' ') {
20             cnt++;
21         }
22         if (cnt == 3) {
23             break;
24         }
25         i--;
26     }
27     //这种输出之前也没有见过，要学习一下，不要再一个一个遍历s再存t了
28     cout << s.substr(0, i + 1) << "qiao ben zhong." << endl;
29 }
30 }

```

5.5.3 最小化思想 2026 天梯选拔赛 1-6

- 题目链接: [1-6 源初之石 \(点击此处\)](#);

代码示例:

```

1 auto solve() -> void {
2     ll n;
3     cin >> n;
4     ll a = -1, b = -1, c = -1;
5     //之所以要看i*i*i是为了找到能使三个数相乘=n的最小值(这里默认把最小值给了a);
6     for (ll i = 2; i * i * i <= n; i++) {
7         if (n % i == 0) {
8             a = i;
9             break;
10        }
11    }
12    if (a == -1) {
13        cout << "NO" << endl;
14        return;
15    }
16    //因为前面的a已经使最小的了所以这里找c的时候可以从a+1开始找，可以提高效率。
17    for (ll i = a + 1; i * i <= n / a; i++) {
18        if ((n / a) % i == 0) {
19            c = i;
20            b = n / a / c;
21            if (b >= 2 && b != a && b != c) {
22                cout << "YES" << endl;
23                cout << a << " " << c << " " << b << endl;
24                return;

```

```
25     }  
26     }  
27     }  
28     cout << "NO" << endl;  
29 }
```

6 AtCoder 刷题总结

在本节中，我将总结我在 AtCoder 上做题的经验。

6.1 遇到的问题以及学到的方法

6.2

- 题目链接：代码示例：



7 AcWing 刷题总结

在本节中，我将总结我在 AcWing 上做题的经验。

7.1 遇到的问题以及学到的方法

7.2 2816 判断子序列

- 题目链接: [2816 判断子序列 \(点击此处\)](#); 代码示例:

```
1 auto solve() -> void {
2     ll n, m;
3     cin >> n >> m;
4     queue<ll> q1, q2;
5     for (ll i = 0; i < n; i++) {
6         ll a;
7         cin >> a;
8         q1.emplace(a);
9     }
10    for (ll i = 0; i < m; i++) {
11        ll b;
12        cin >> b;
13        q2.emplace(b);
14    }
15    while (q1.size() && q2.size()) {
16        if (q1.front() == q2.front()) {
17            q1.pop();
18            q2.pop();
19        } else {
20            q2.pop();
21        }
22    }
23    if (q1.size() == 0) {
24        cout << "Yes" << endl;
25    } else if (q2.size() == 0) {
26        cout << "No" << endl;
27    }
28 }
```